

Software Engineering

How To Create Software Products Everyone Loves!

Module 15

Software Architecture and Technical Design

Ali Samanipour

May. 2023

Ali Samanipour
[linkedin.com/in/Samanipour](https://www.linkedin.com/in/Samanipour)

1

Overview of Software Process

2

Modeling In Software Engineering

3

Cornerstones of Software Engineering

4

Review of 14 Unified Modeling Language Diagrams

5

Modeling Spaces in Software Engineering

6

Design and Modeling Tools

A software production line!

1.
Plan



Product
owner

Product
manager



2.
Analysis



Project
manager

Software
architecture



3.
Design

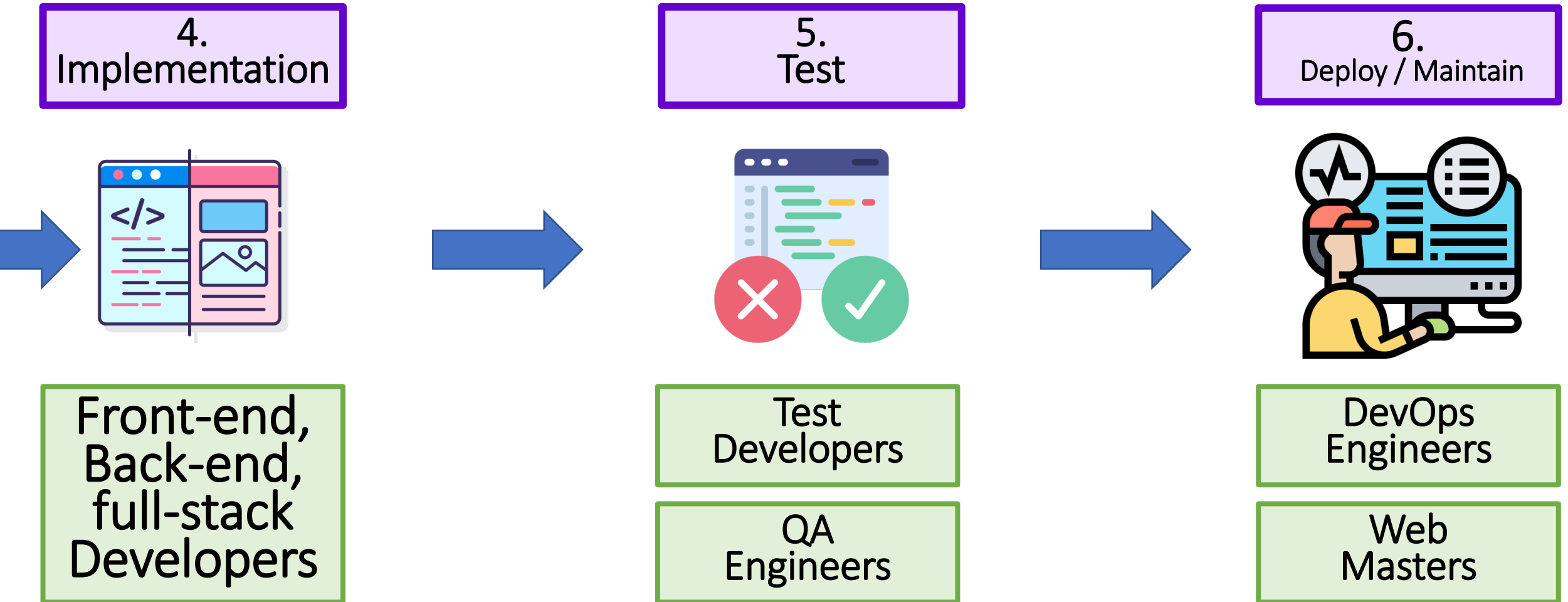


UI/UX
Designer

Technical
Designer

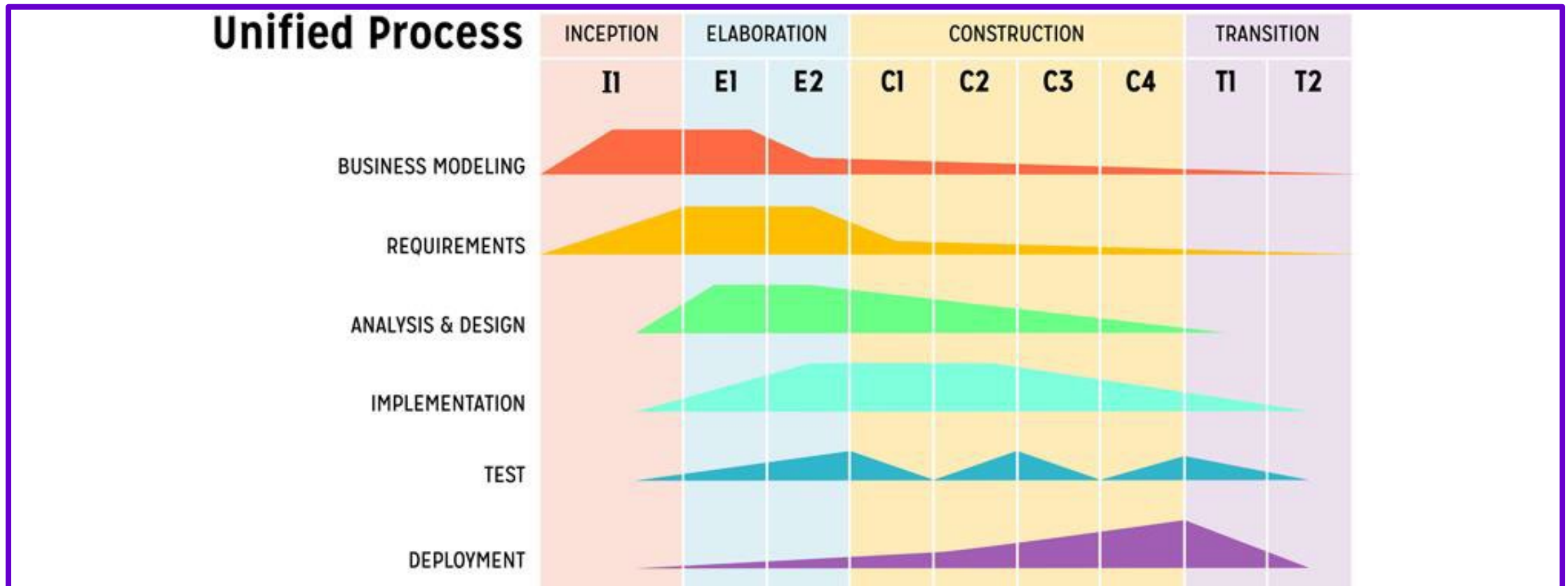


A software production line!



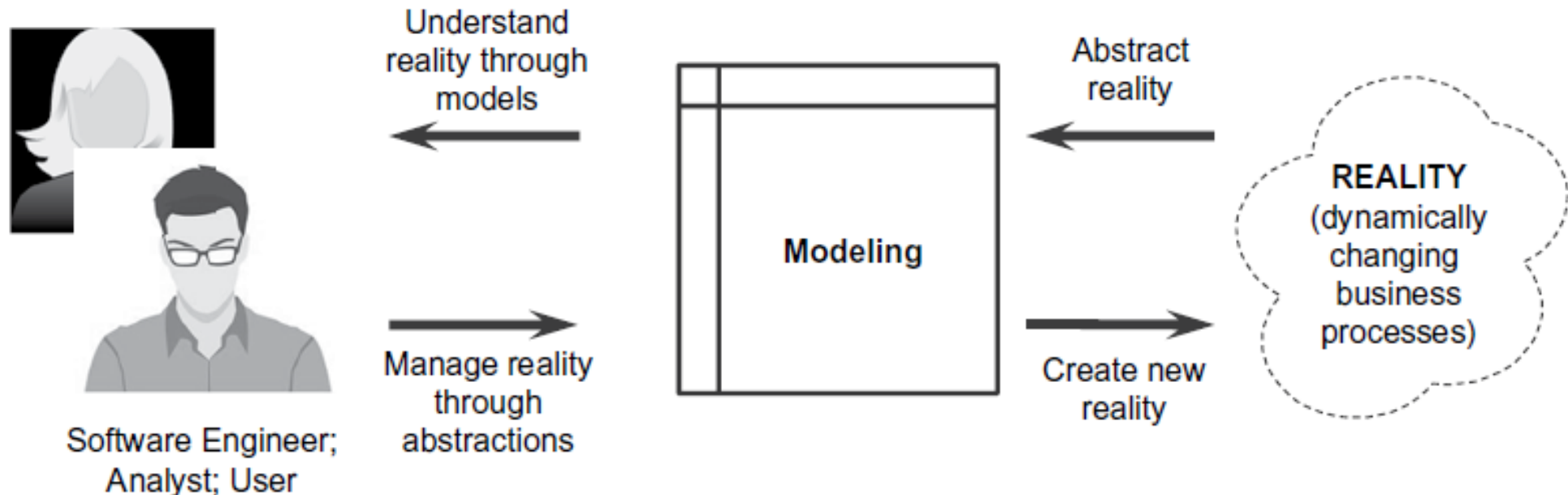
Parallel Models: Unified Process Model

The **Unified Process** basic structure has sequential phases within a repeatable cycle.



Importance of modeling in software engineering: Understand and create reality

Modeling enables a much better **understanding of a problem** and the **solution before** the solution is coded—thereby reducing unnecessary efforts and improving the quality of software.



Reality vs Model

A model is a representation of reality, but it is a simplification or approximation that doesn't reflect all of reality (**Model is an Abstraction of Reality**)

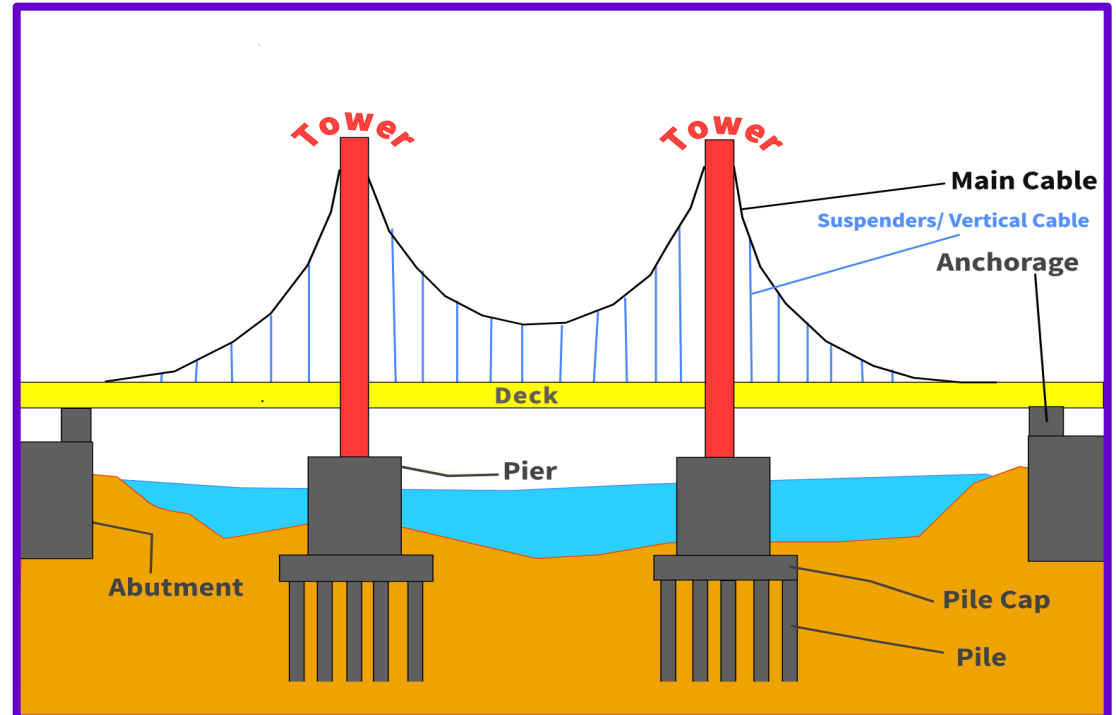
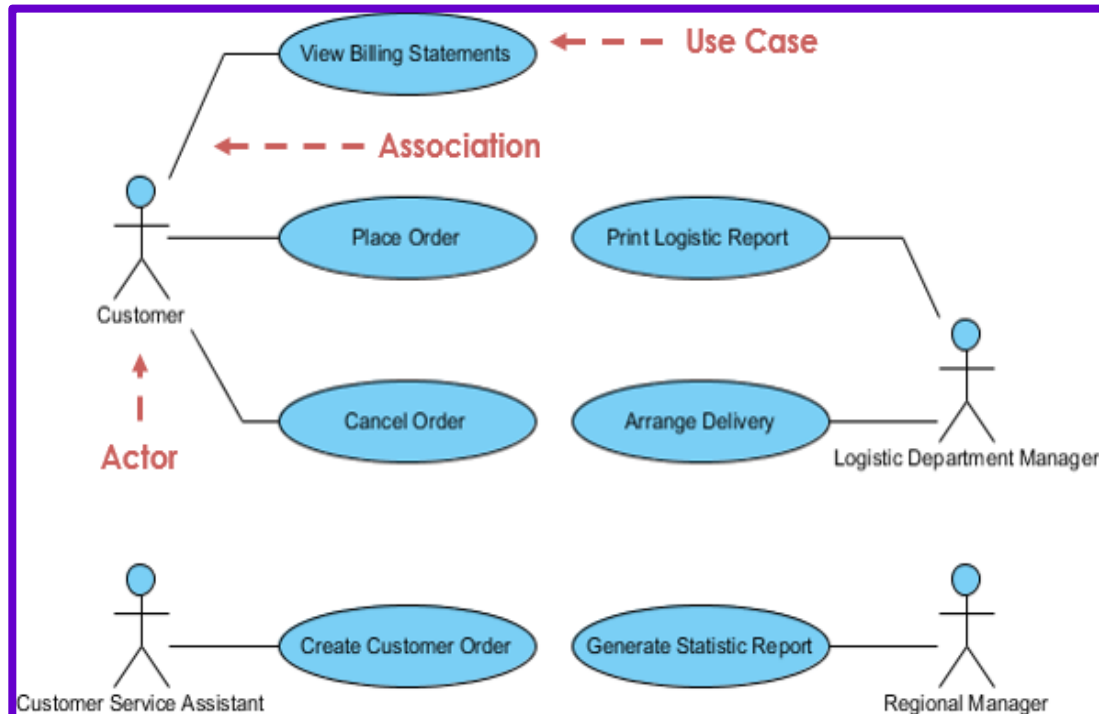
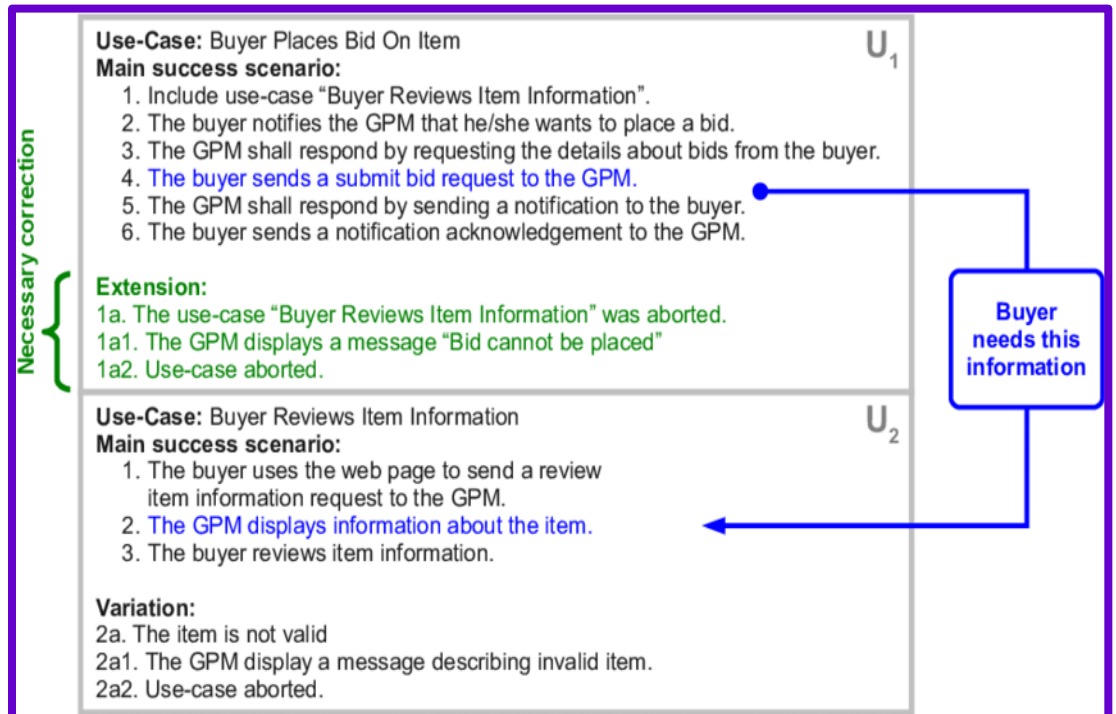


Diagram vs Model

Diagram is a **Visual Representation** of Model

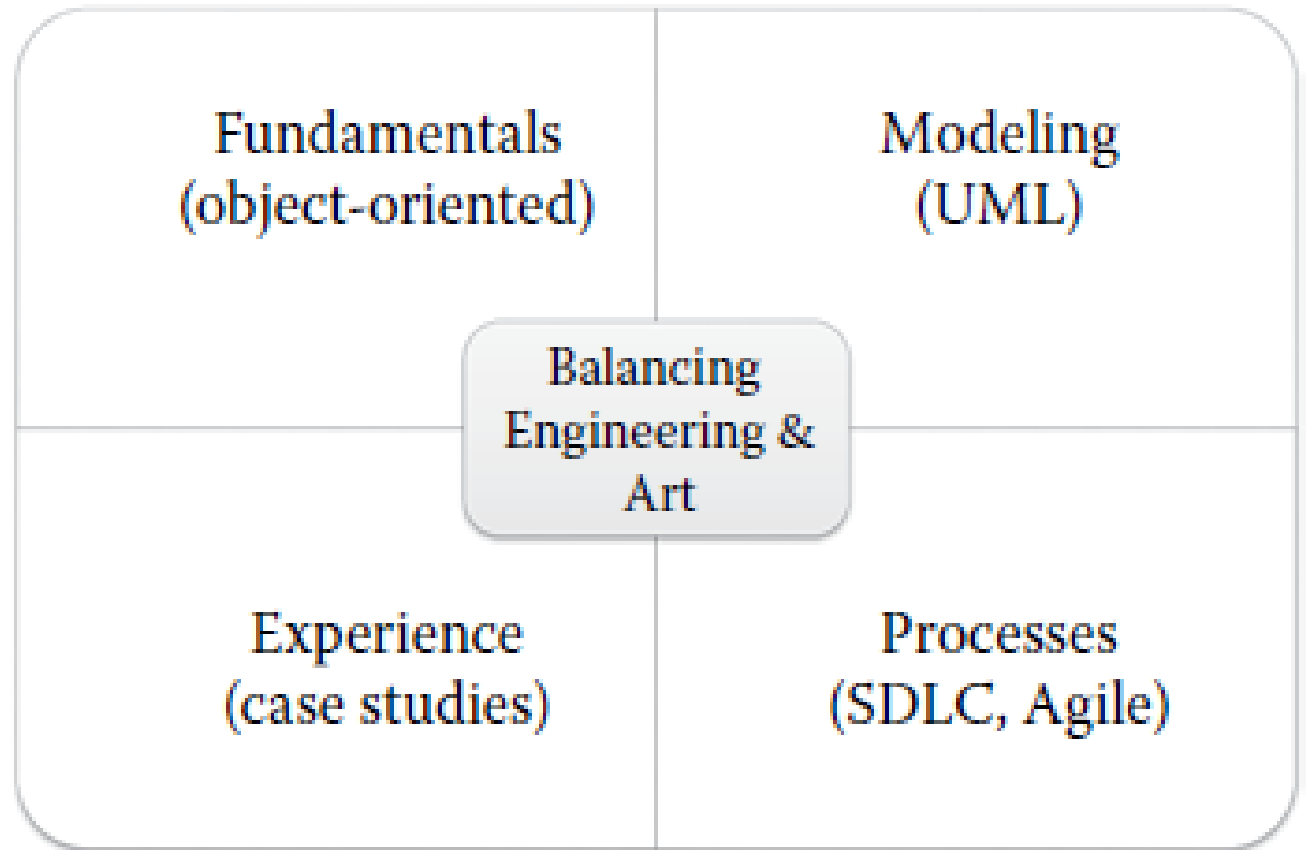


Models are **abstract representations** of a system



Learning and Adopting Software Engineering

Experience in
creating UML
models, especially
in a team
environment, is a
must for learning
the art of SE



Overview of Basic Concept

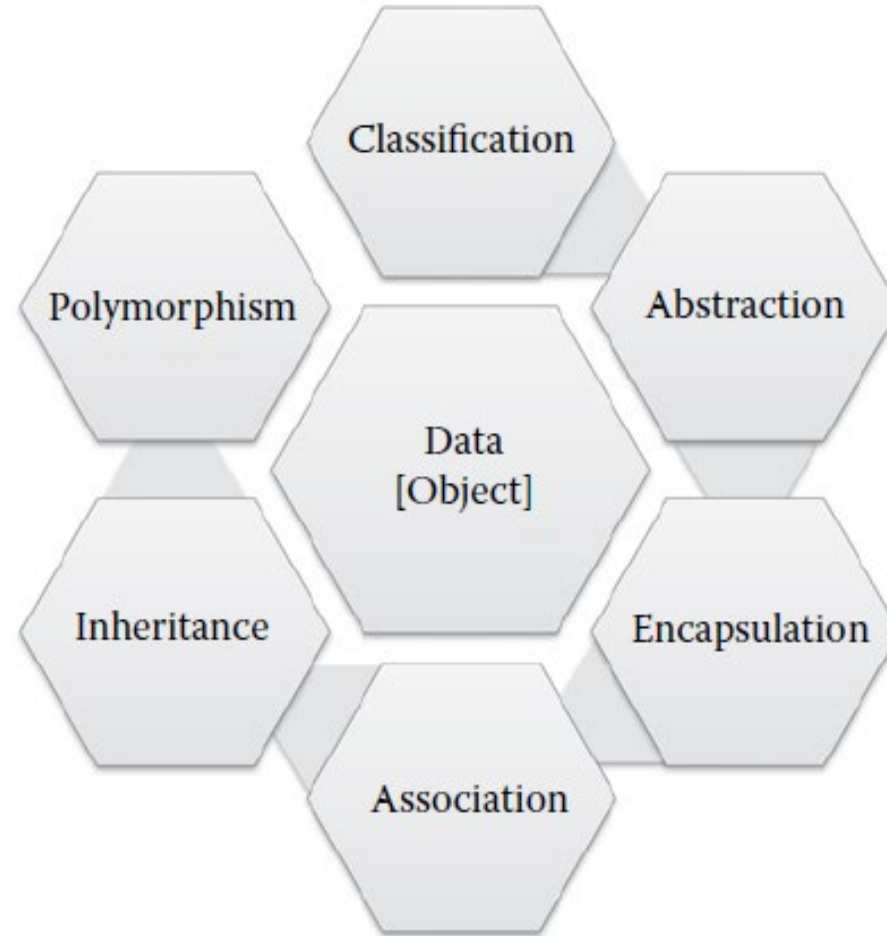
Programs—structured code specific to a programming language that reflect an algorithm

Classes—specific styles of software programs that encapsulate data with functions (methods)

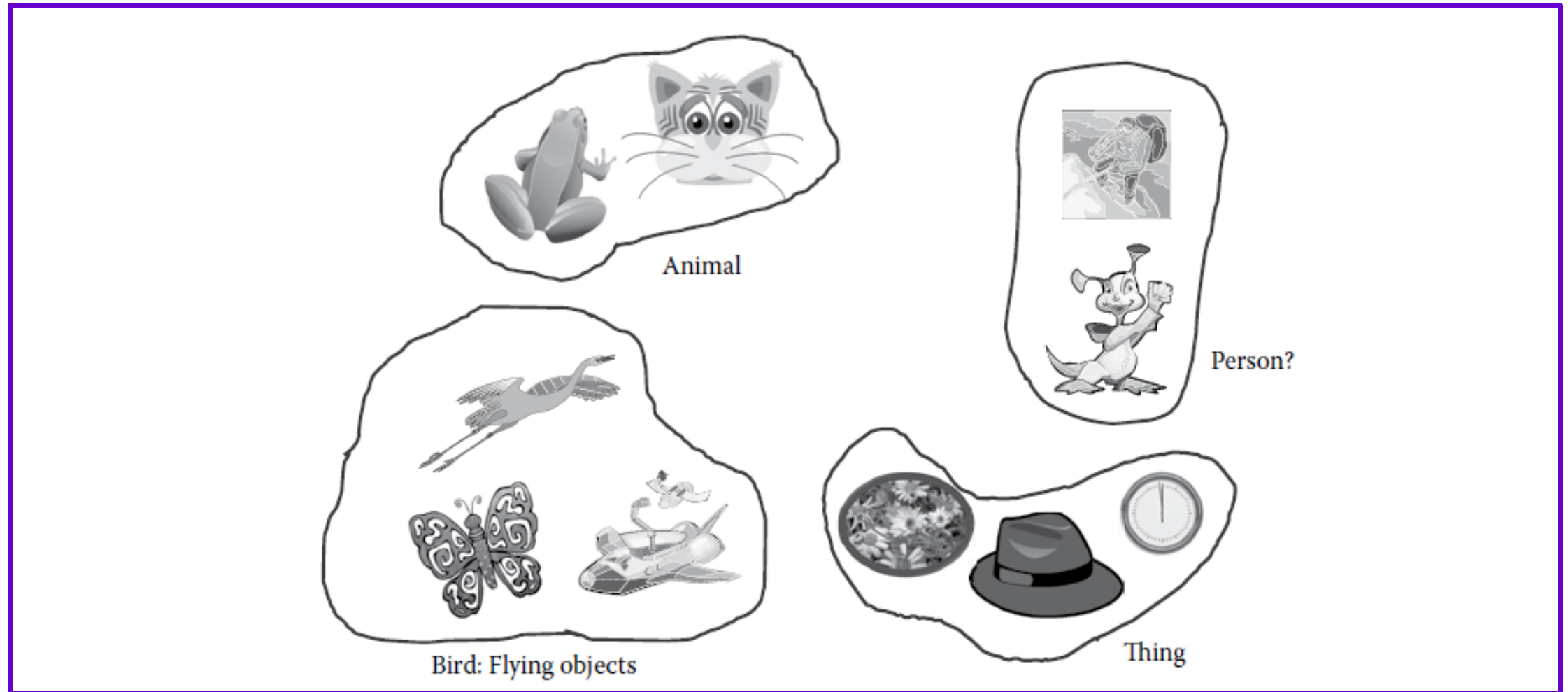
Data—instance-level representation of the business reality (characteristics) that is encoded and stored within databases

Objects—instances of classes that comprise both data and the behavior associated with the data at runtime.

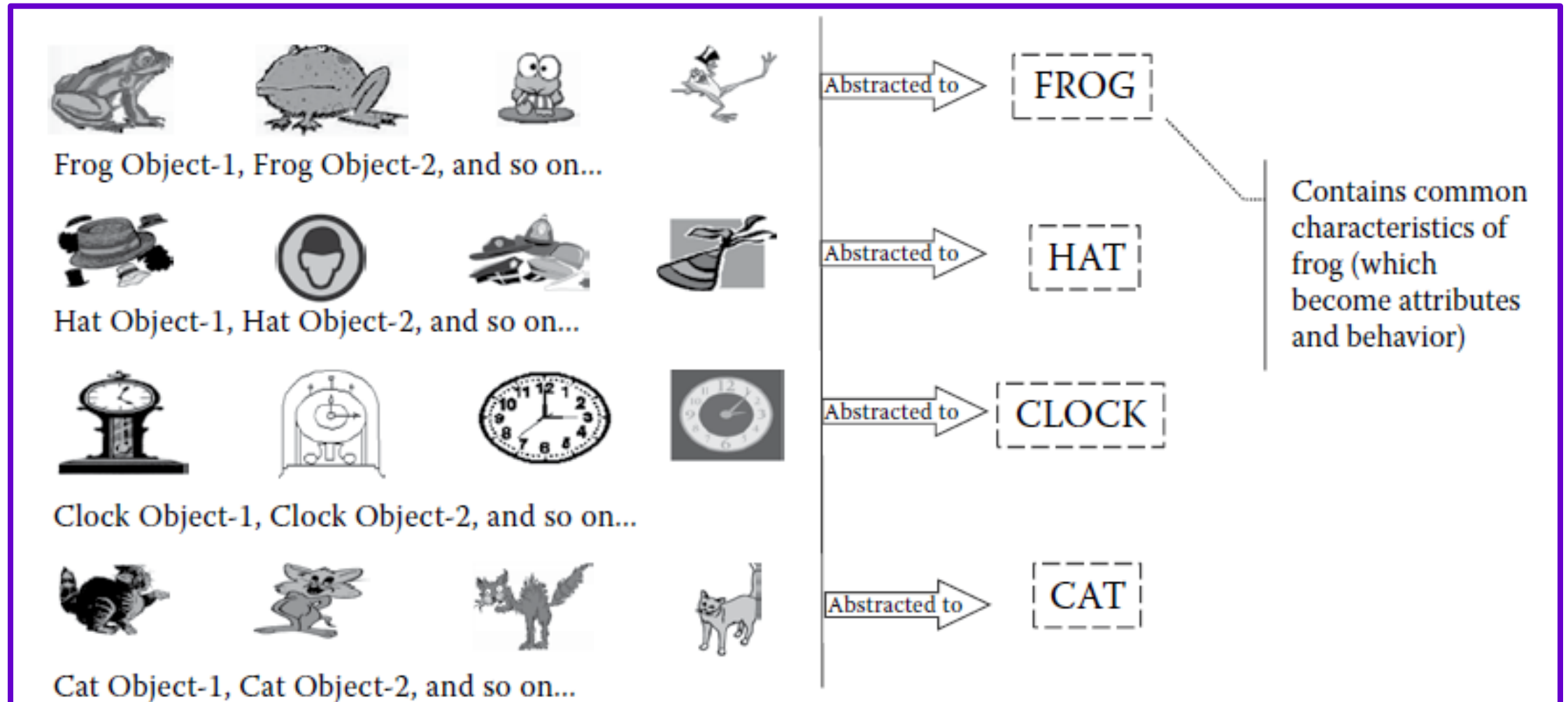
The Six Fundamentals (Cornerstone) of Software Engineering



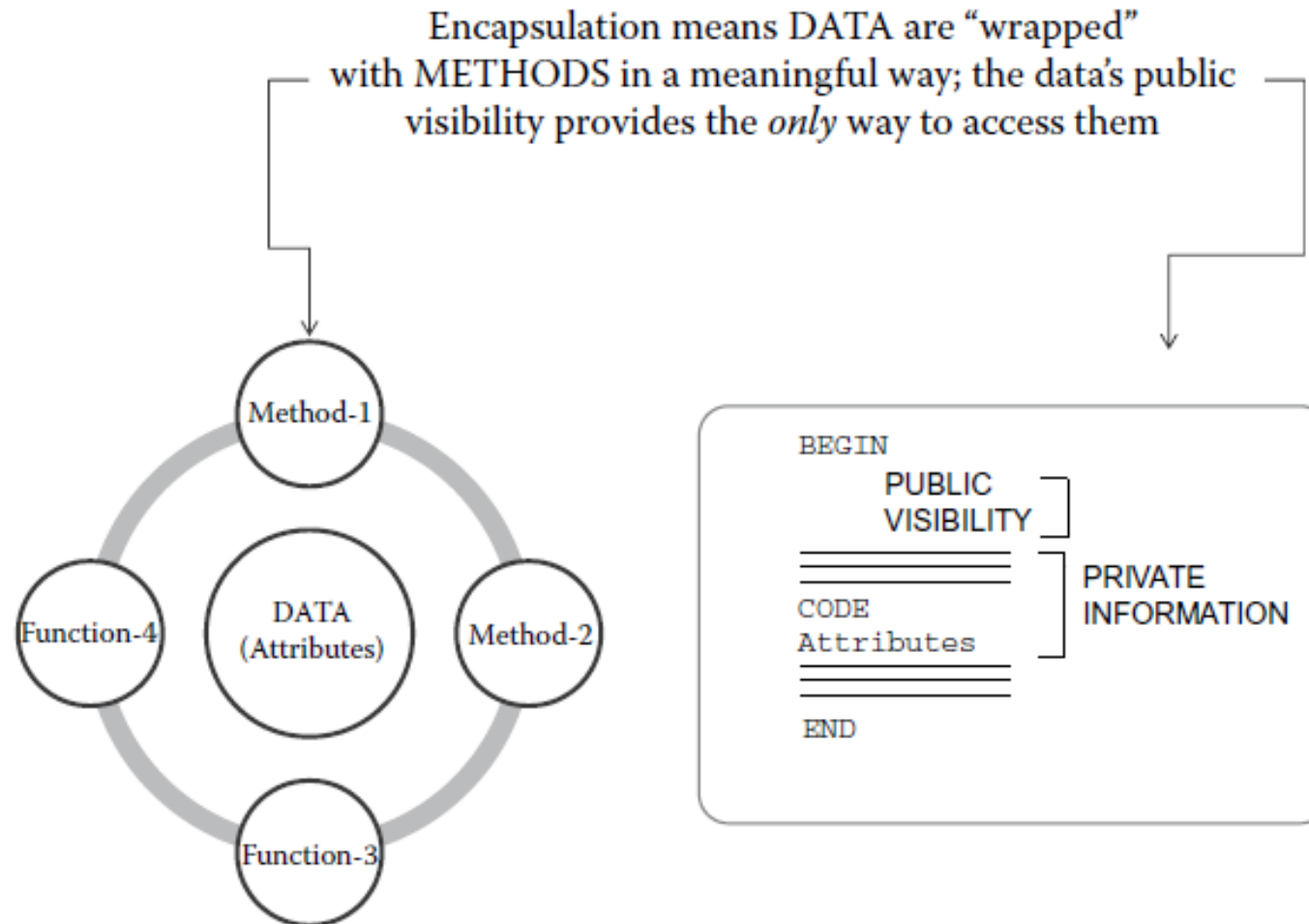
Classification (Grouping)



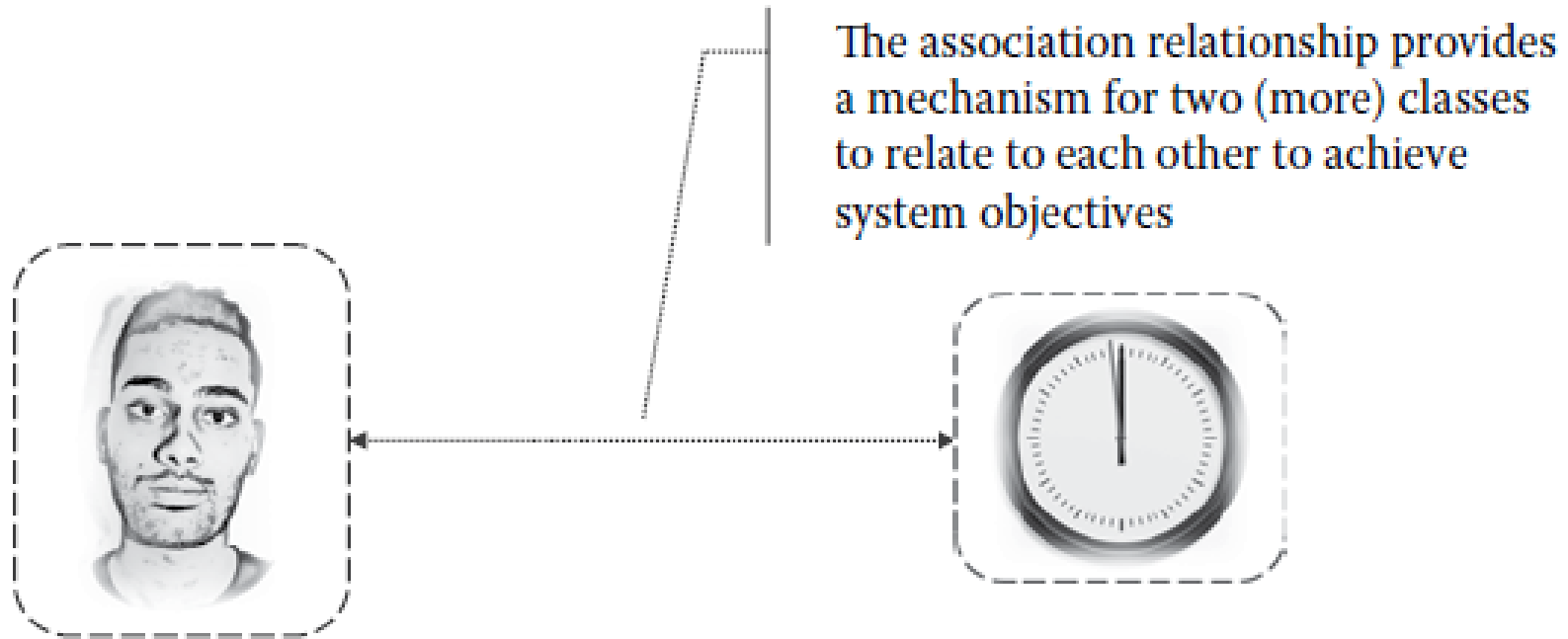
Abstraction (Representing)



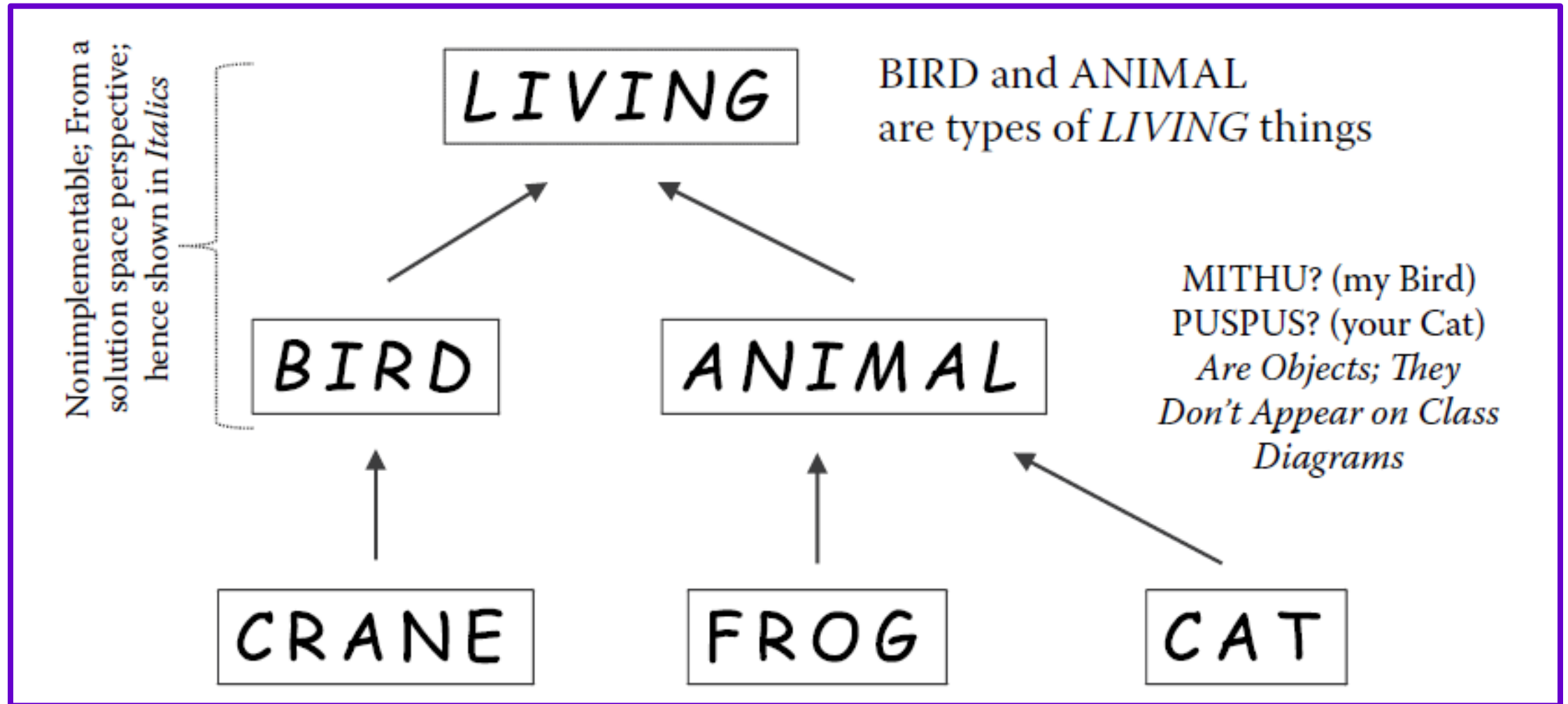
Encapsulation (Modularizing)



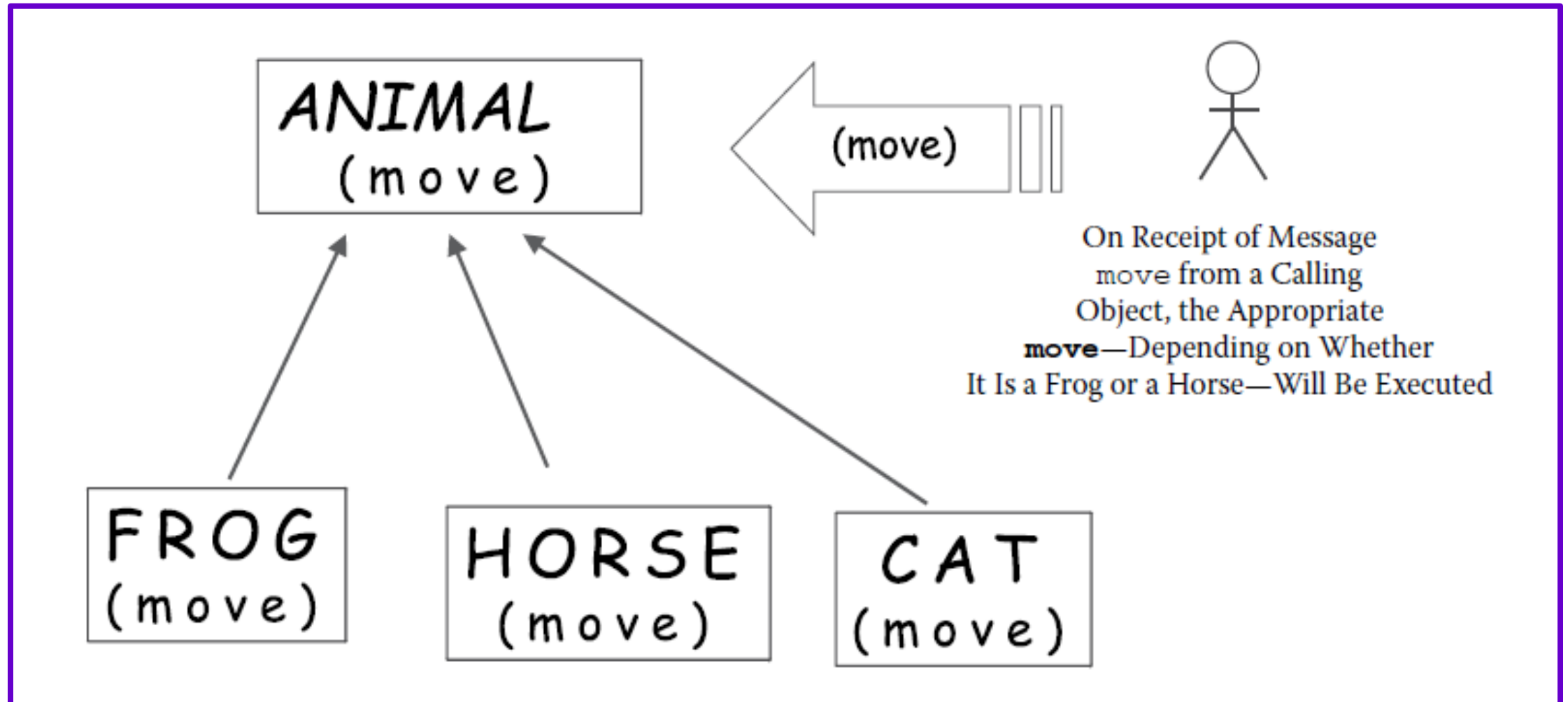
Association (Relating)



Inheritance (Generalizing)



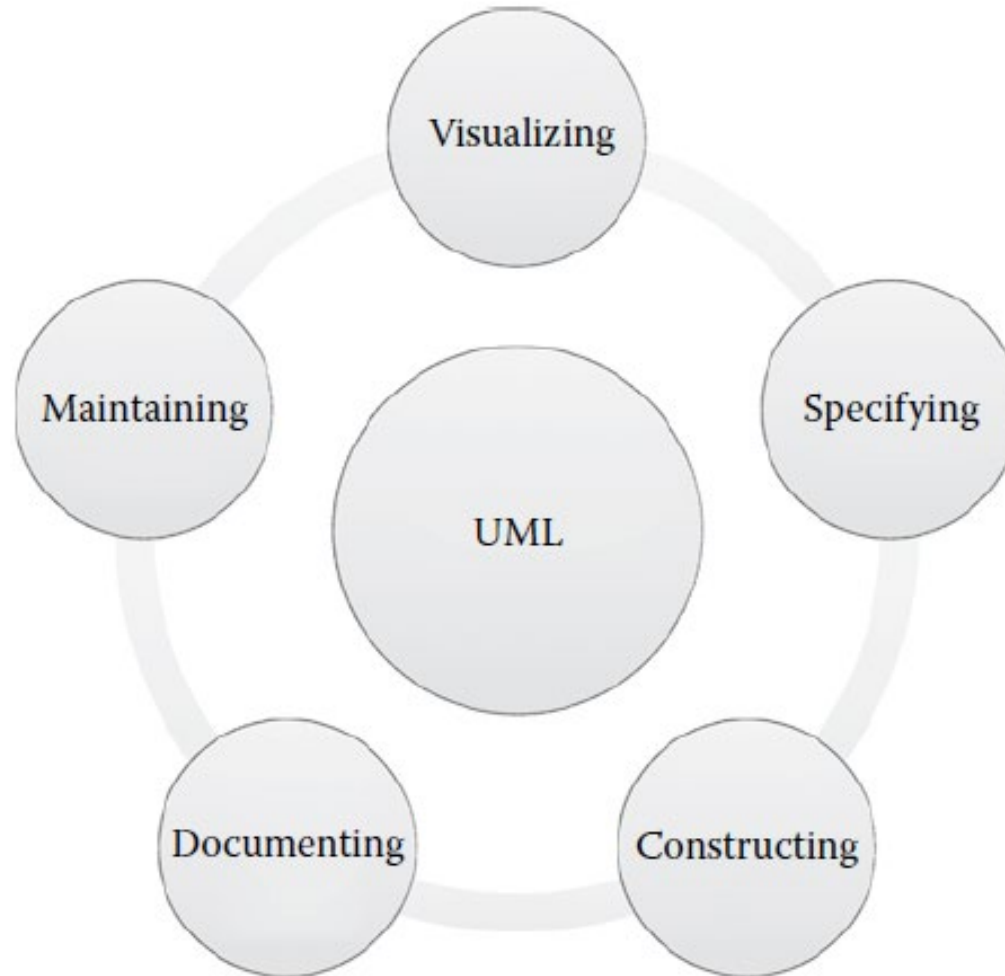
Polymorphism (Executing)



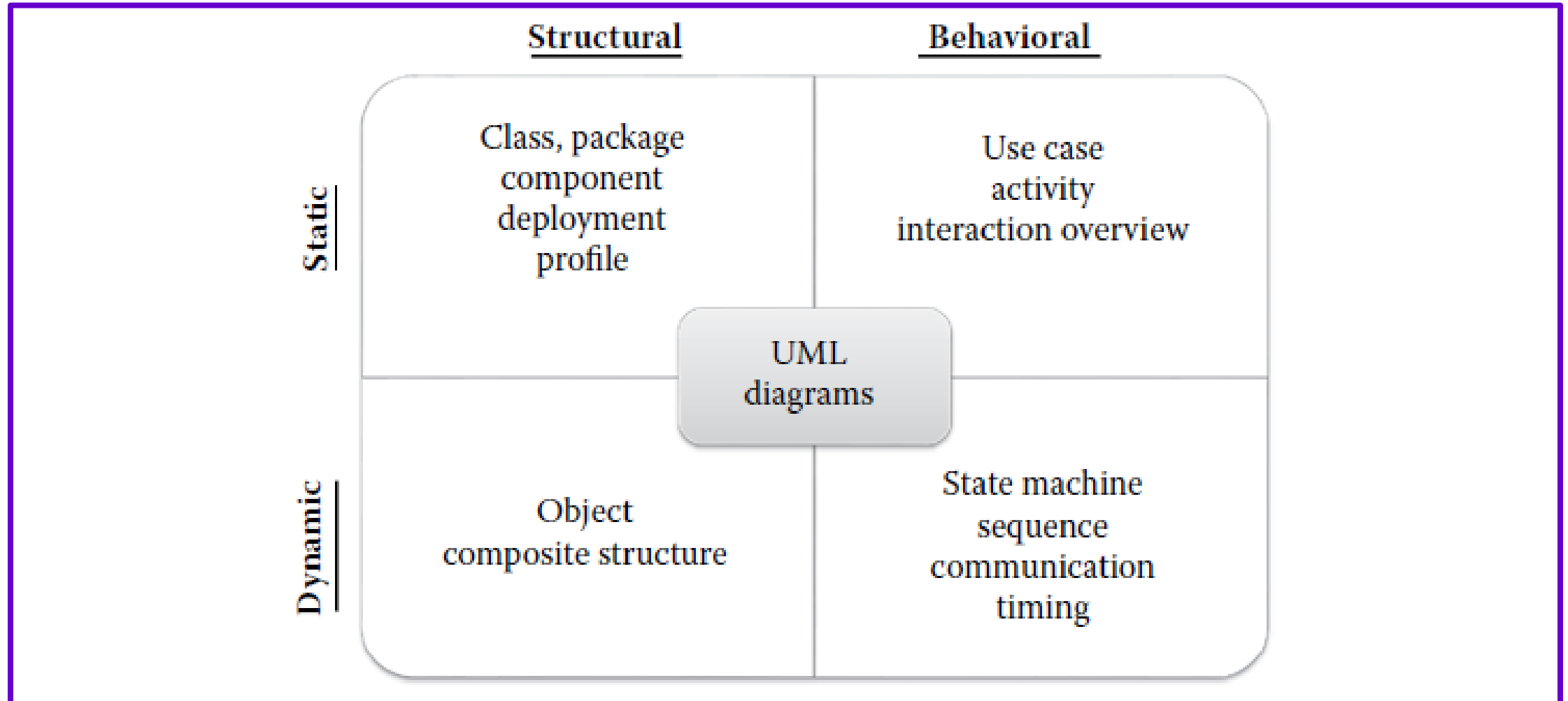
What is UML?

Is a **standardized modeling language** consisting of an **integrated set of diagrams**, developed to help system and software developers **for specifying, visualizing, constructing, and documenting the artifacts** of software systems, as well as for business modeling and other non-software systems

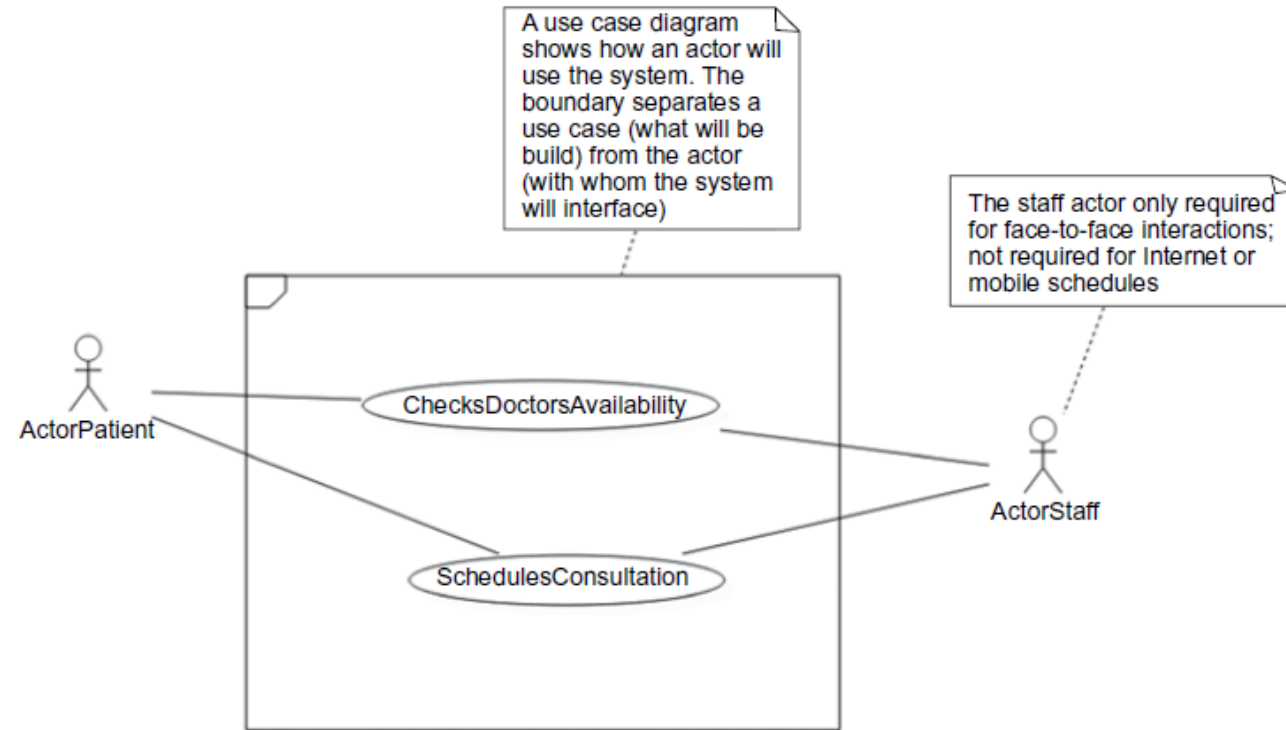
UML and Its Purpose



UML Diagrams



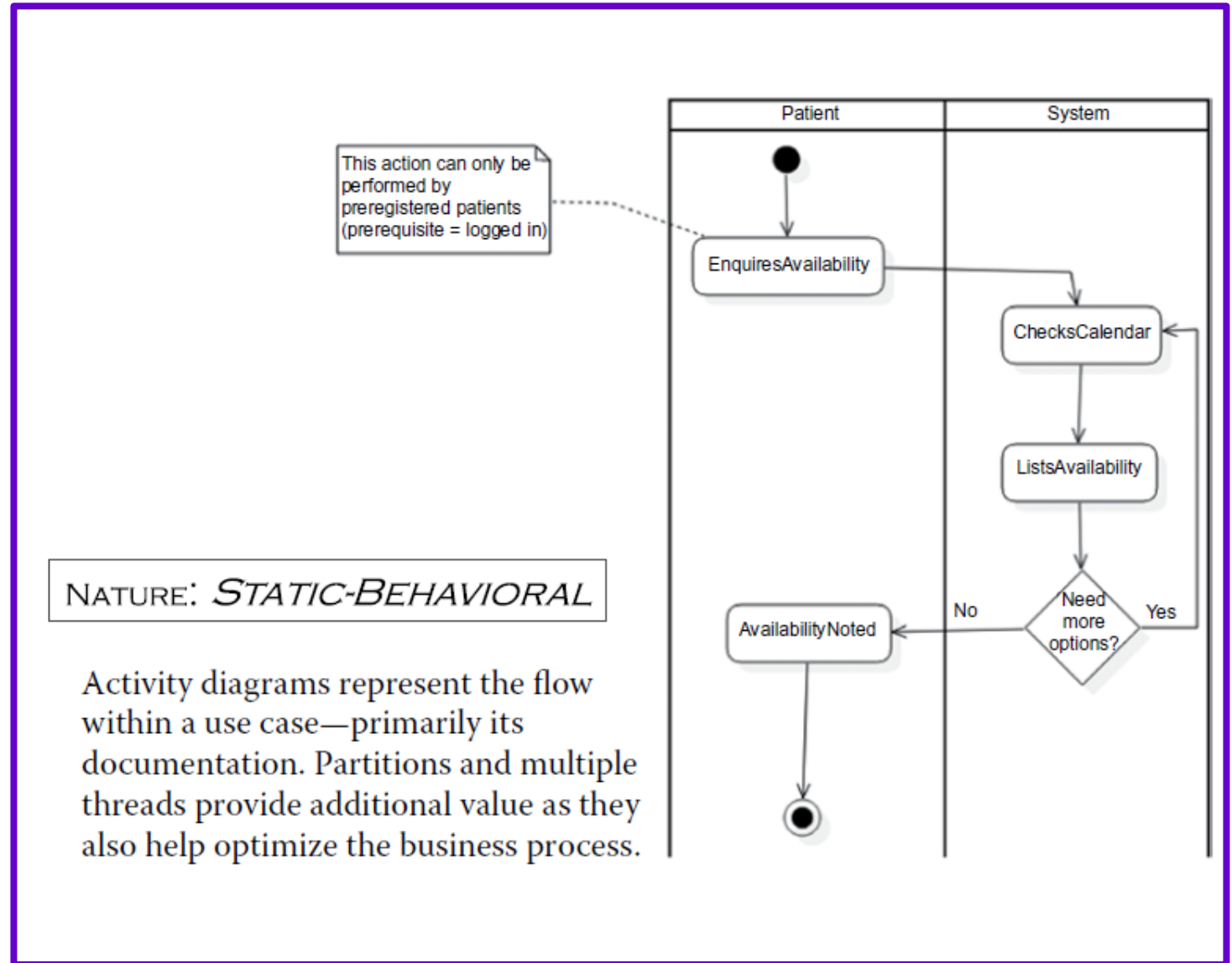
Use case diagram



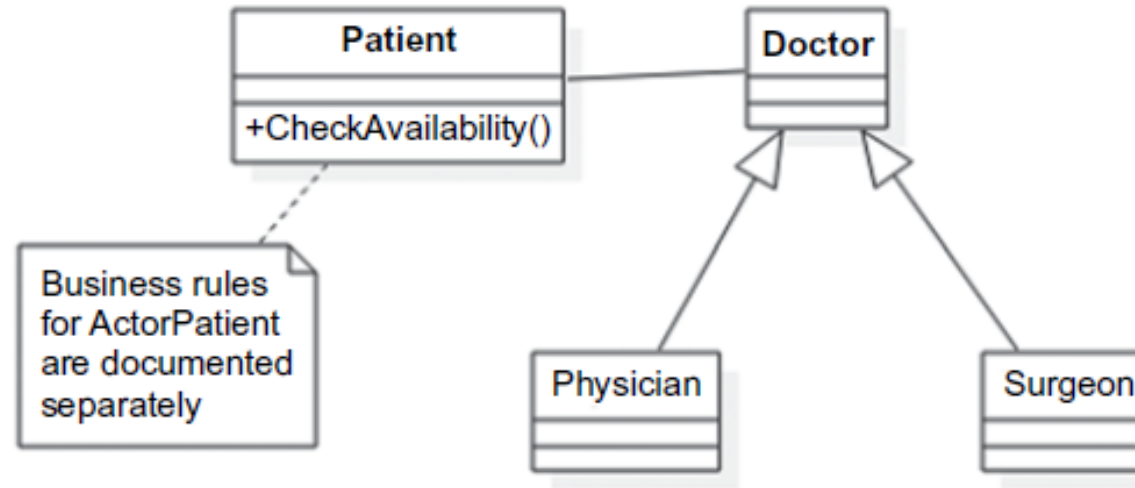
NATURE: *STATIC-BEHAVIORAL*

Use case diagrams providing an overview of the requirements through actors and use cases. Internal documentation of the use cases contains details of the interactions between actor and system.

Activity Diagrams



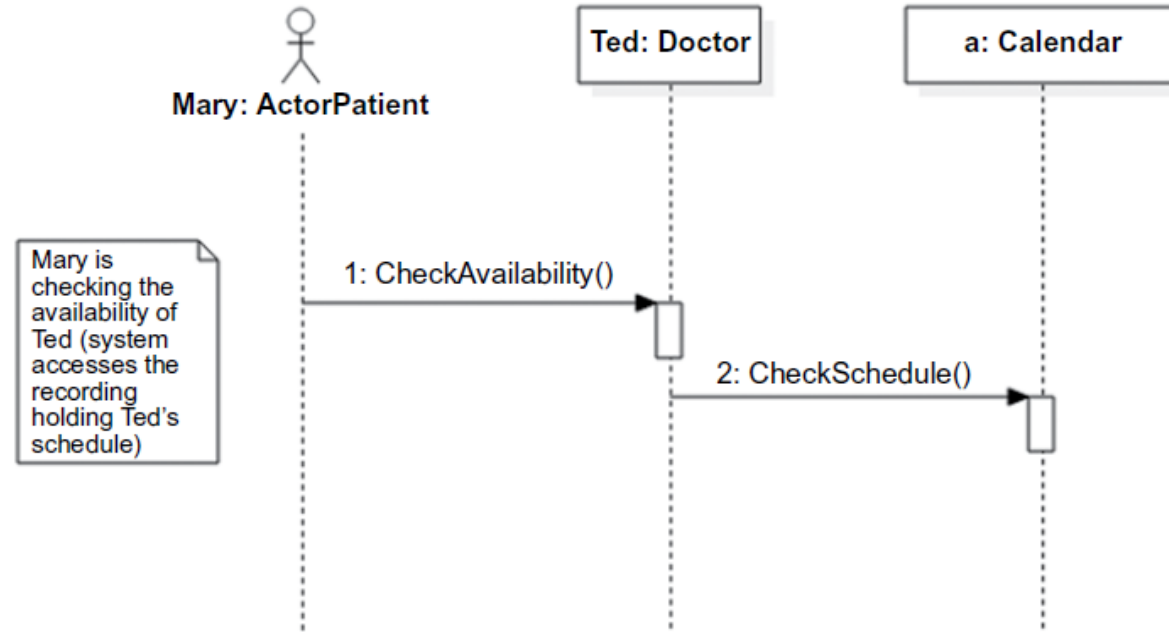
Class Diagrams



NATURE: *STATIC-STRUCTURAL*

Class diagrams model entities (i.e., classes at business and technical levels) and their relationships. Classes in these diagrams contain attributes and operations (which can be visible or hidden), relationships (inheritance, association), and multiplicities.

Sequence diagrams



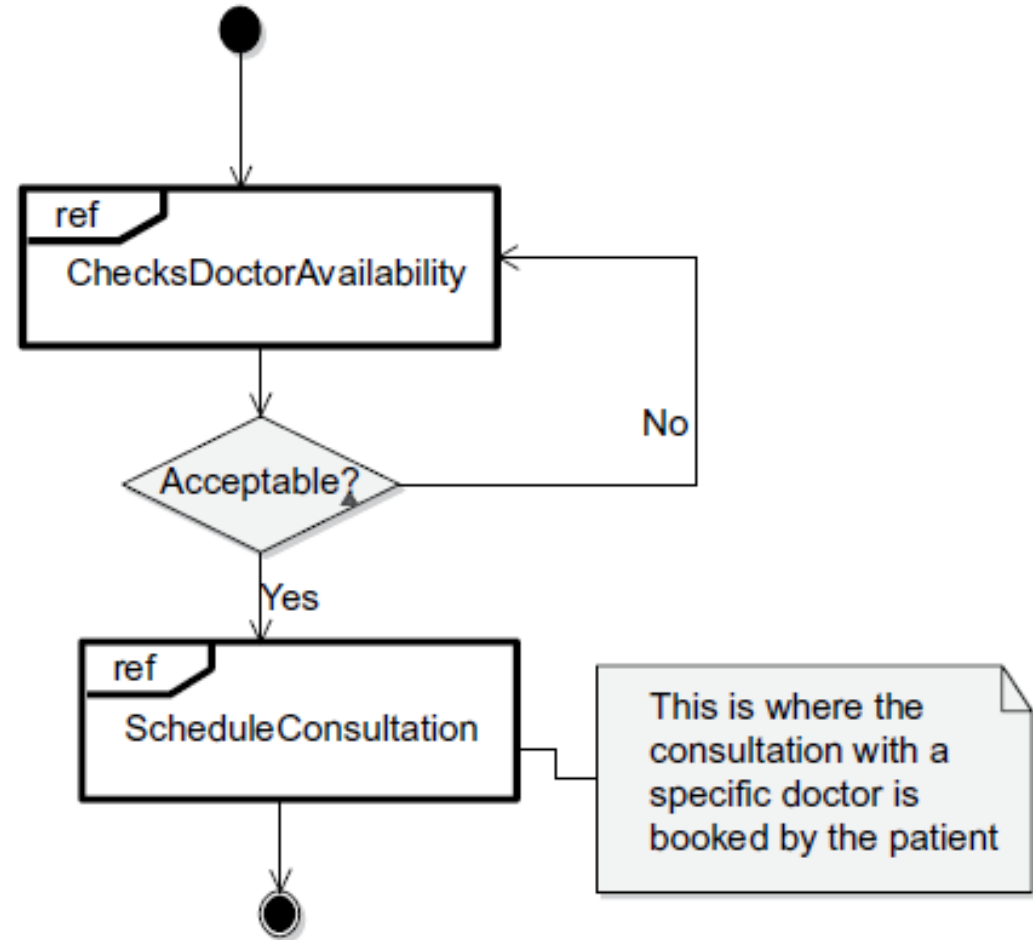
NATURE: *DYNAMIC-BEHAVIORAL*

Sequence diagrams show a single scenario of interactions between objects and system (through messages). The sequence of messages is important. These diagrams may contain actors. Sequence diagrams cannot show conditions ("if-then-else").

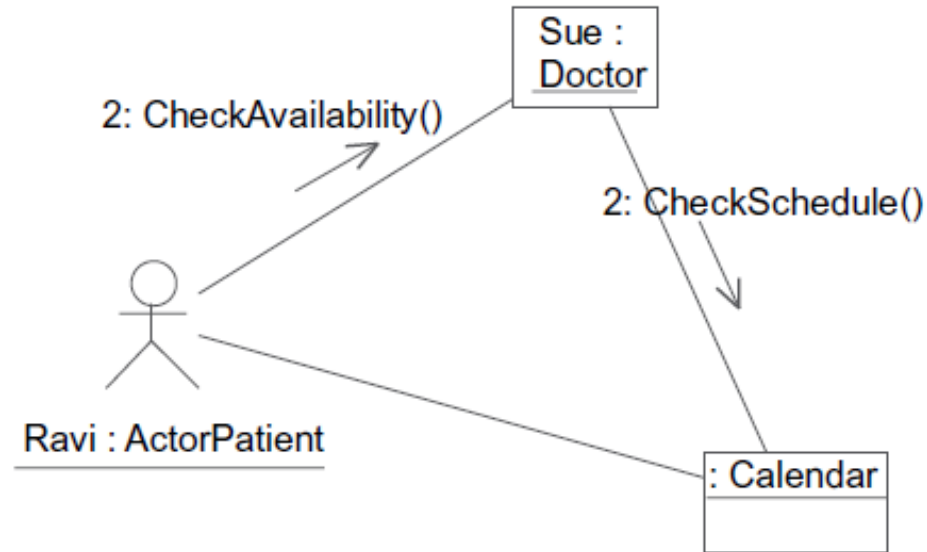
Interaction Overview Diagrams

NATURE: *STATIC-BEHAVIORAL*

Interaction overview diagrams provide a high-level overview of how other diagrams—such as sequence diagrams or even use cases—are related to each other; they reference interactions; conceptually, they are similar to activity diagrams (because they have a flow within them)



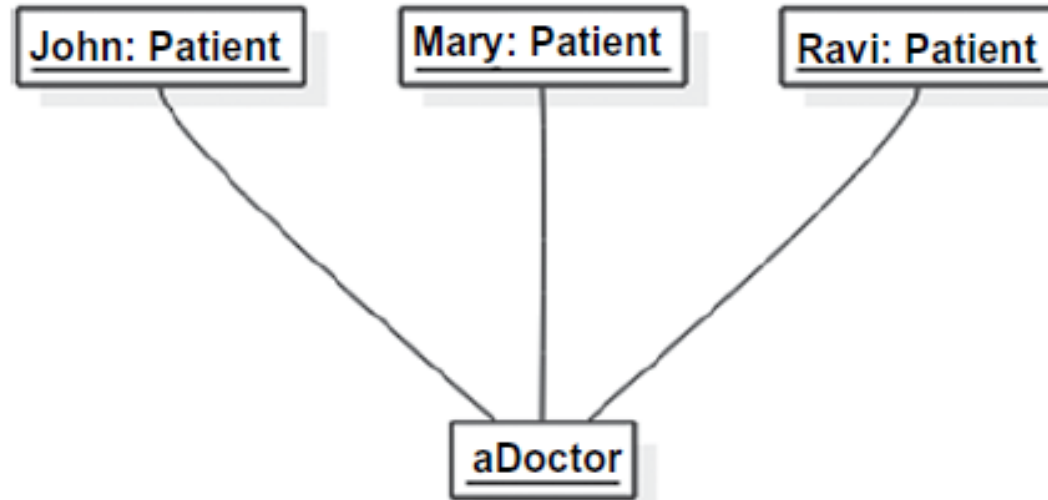
Communication Diagrams



NATURE: *DYNAMIC-BEHAVIORAL*

Communication diagrams are an alternative view to the sequence diagrams. These diagrams model interactions between objects and their links to each other. The sequencing of messages is depicted by numbers.

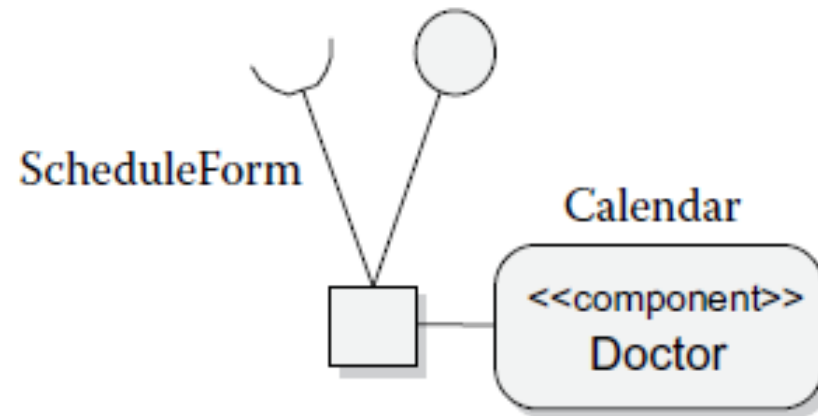
Object Diagram



NATURE: DYNAMIC-STRUCTURAL

Object diagrams describe the various objects (*instances*) and how they relate to each other. The relationships are *links* in the memory. Being instance-level diagrams, they are ideal in depicting the multiplicities between classes.

Composite structure diagram



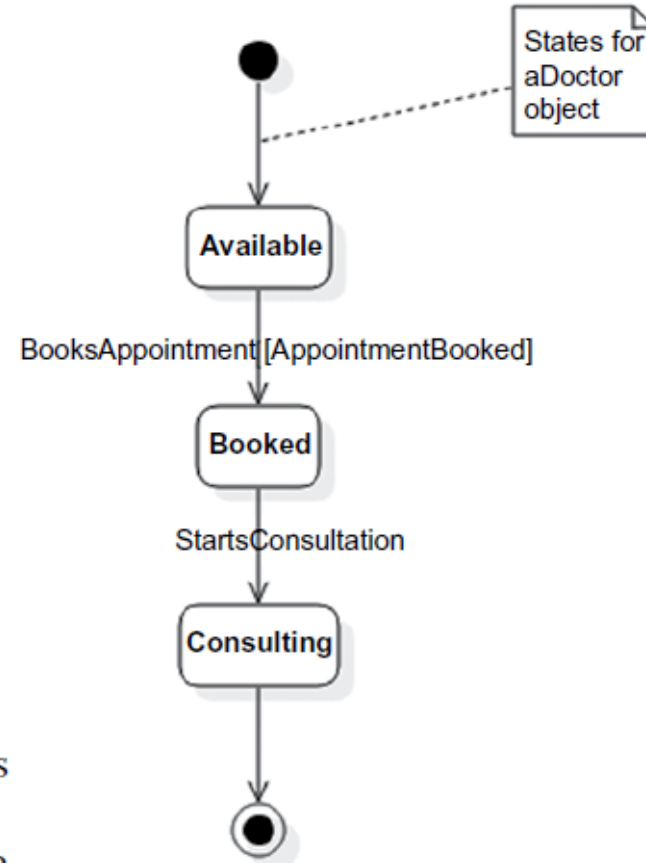
NATURE: *DYNAMIC-STRUCTURAL*

Composite structure diagrams show links and decompositions of components as well as objects at runtime in memory

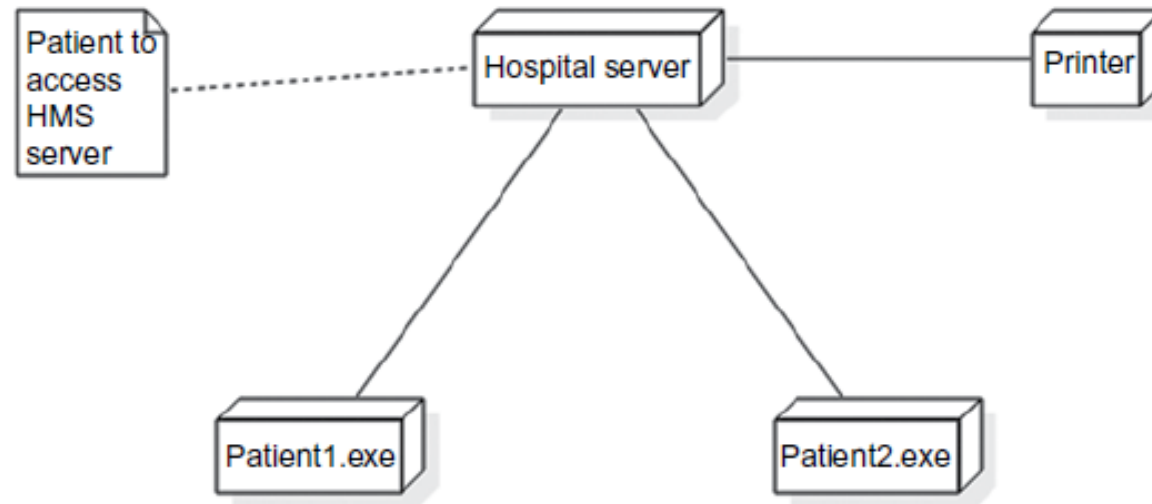
State machine diagram

NATURE:
DYNAMIC-BEHAVIORAL

State machine diagrams show the various states of an object; they also show the events and guard conditions under which a change in state occurs for an object. They are usually referred to by their corresponding class name.



Deployment diagram



NATURE: *STATIC-STRUCTURAL*

A deployment diagram shows the manner in which a system will be deployed when in operation. These diagrams show processes and nodes in the physical design of a system. They are the only hardware diagram in the UML

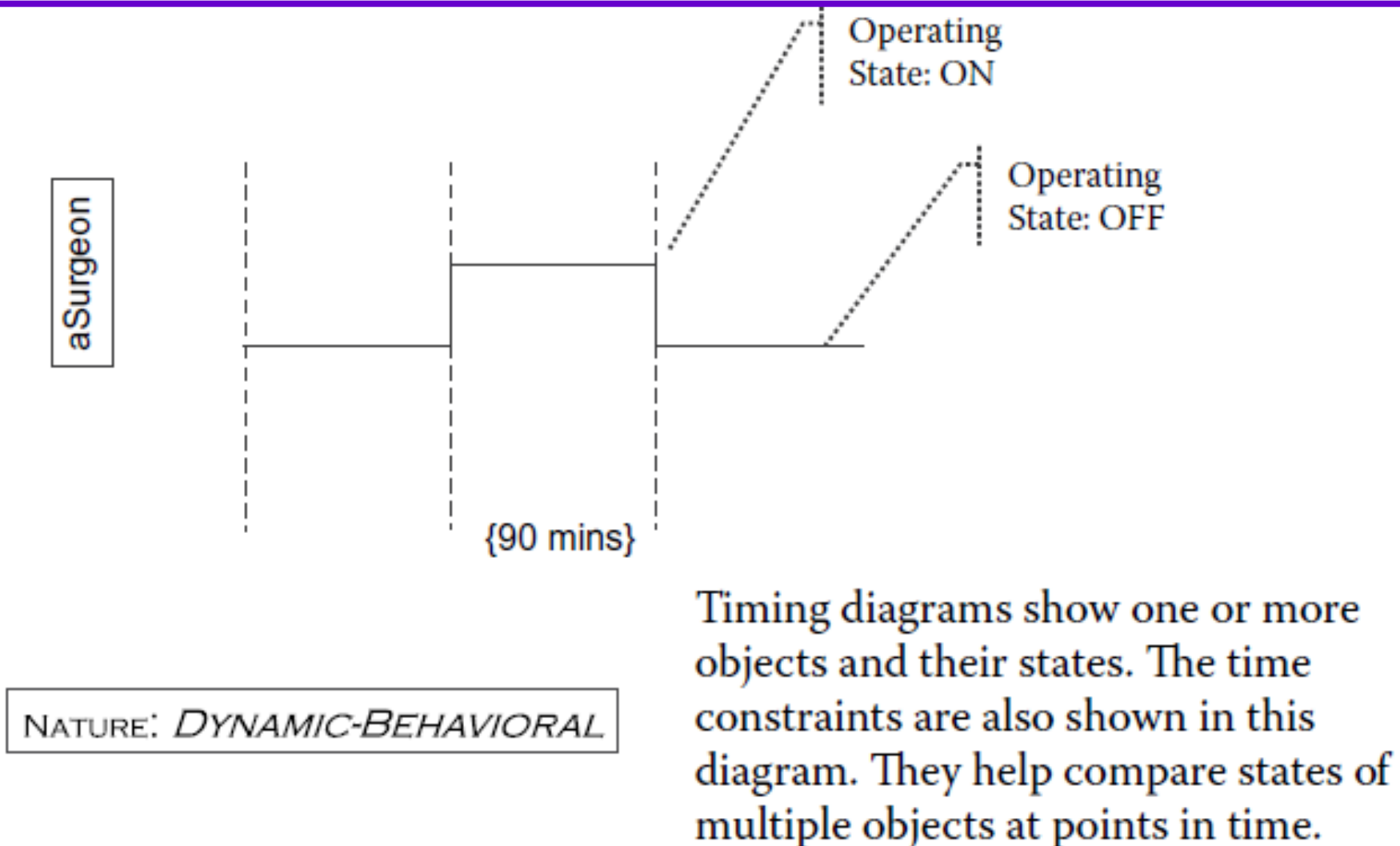
Package diagram



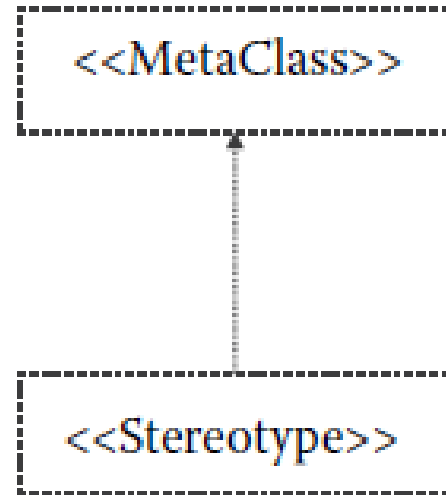
NATURE: *STATIC-STRUCTURAL*

Packages represent subsystems; They comprise large and cohesive collections of other UML diagrams. Package diagrams are organizational in nature and show packages and their dependencies.

Timing diagram



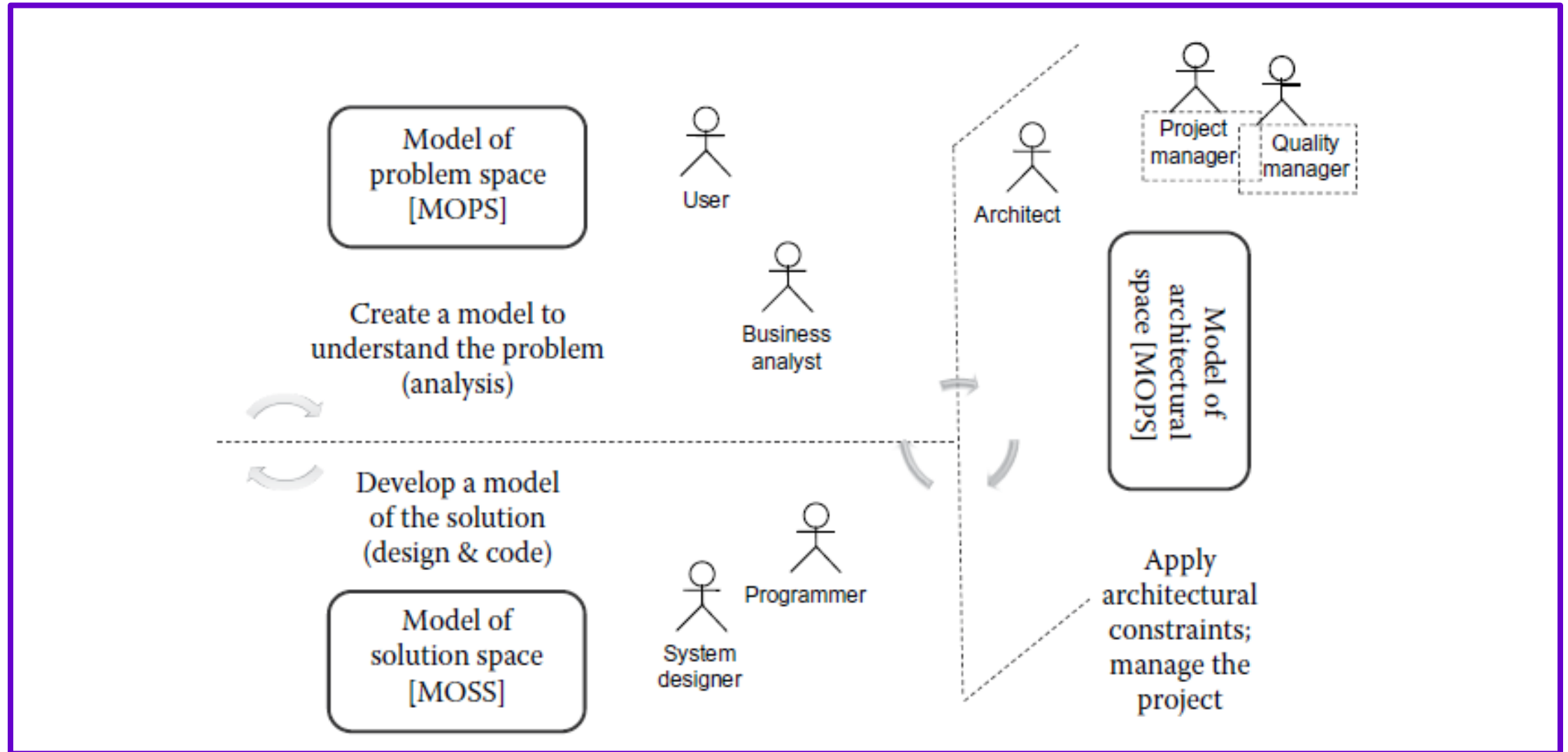
Profile diagram



NATURE: *STATIC-STRUCTURAL*

Profile diagrams enable extension of UML such as visualization of stereotypes (e.g., inheriting all the characteristics of a Meta-class as shown in this diagram)

Modeling Spaces in Software Engineering



Importance of UML Diagrams to Respective Models

With this understanding of the three modeling spaces, it is now easier to understand how each of the 14 UML diagrams can play a role in these different modeling spaces with varying degrees of importance and relevance.

<i>UML Diagrams</i>	<i>MOPS (Business Analyst)</i>	<i>MOSS (Designer)</i>	<i>MOAS (Architect)</i>
Use case	*****	**	*
Activity	*****	**	*
Class	***	*****	**
Sequence	****	*****	*
Interaction overview	****	**	**
Communication	*	***	*
Object	*	*****	***
State machine	***	****	**
Composite structure	*	*****	****
Component	*	***	*****
Deployment	**	**	*****
Package	***	**	****
Timing	*	***	***
Profile	*	**	****

General Design and Modeling Tools: Draw.io

 draw.io

Blog

Start Now

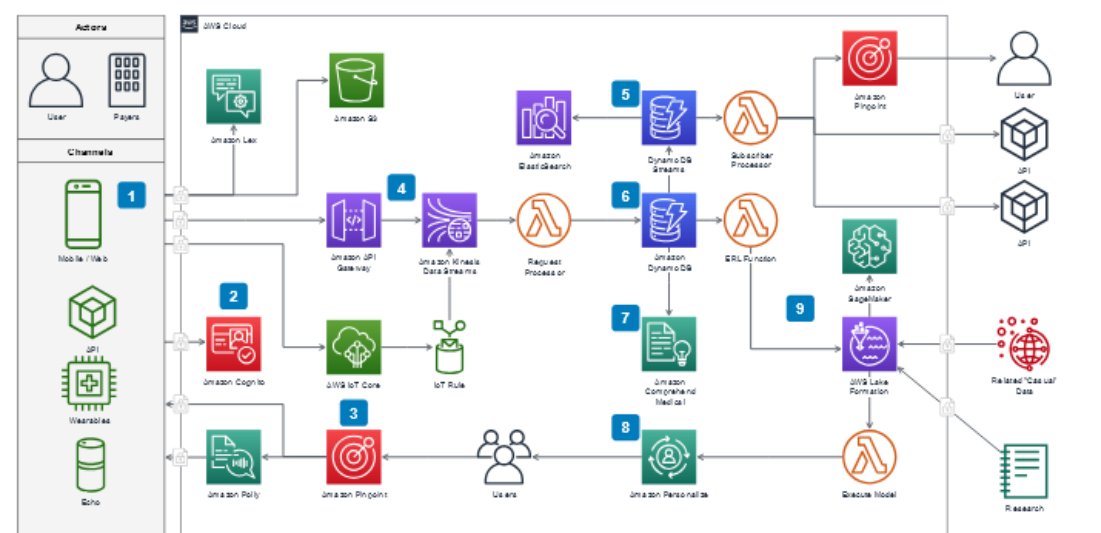
Security-first diagramming for teams.

Bring your storage to our online tool, or save locally with the desktop app.

Start

Download

No login or registration required.



General Design and Modeling Tools: Visio

The image shows the Microsoft Visio website interface. The top navigation bar includes the Microsoft logo, "Microsoft 365", "Visio", "Plans and pricing", "Features", and "Resources". On the right, there are links for "All Microsoft", a search bar, and a "Sign in" button. The main content area features the "Visio" logo and the tagline "Work visually from anywhere, at any time." Below this are two buttons: "See plans and pricing" and "Sign in".

To the right of the text is a laptop displaying the Visio application. The application window shows a flowchart titled "Contoso Environmental Clearance". The flowchart starts with a "Proposed Action" shape, leading to a decision diamond "Are impacts significant?". If "Yes", it goes to a "Decision Notice" shape. If "No", it goes to a "Public Comment" shape, then to a "Public Consultation" shape, then to a "Final Environmental Impact Statement" shape, and finally to a "Record of Decision" shape. There is also a "Start/End" shape at the beginning and end of the flow. The left sidebar of the Visio application shows a "Shapes" pane with various flowchart symbols like Process, Decision, Subprocess, Start/End, Document, Data, Database, External Data, and Custom shapes. The right sidebar shows a "Comments" pane with a "New Comment" button and a user profile for "Filip Sufanek".

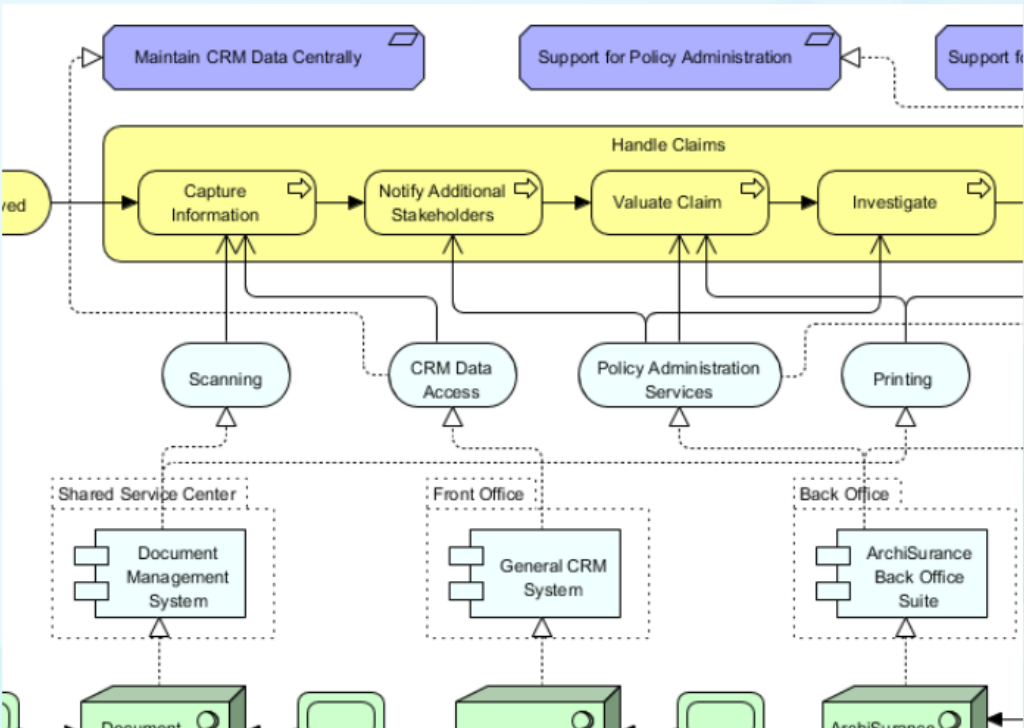
Specialized Design and Modeling Tools: Visual Paradigm

[What's New](#) [Features](#) [Tutorials](#) [Support](#) [Pricing](#) [Try Now](#) [Request Demo](#) [VP Online](#)

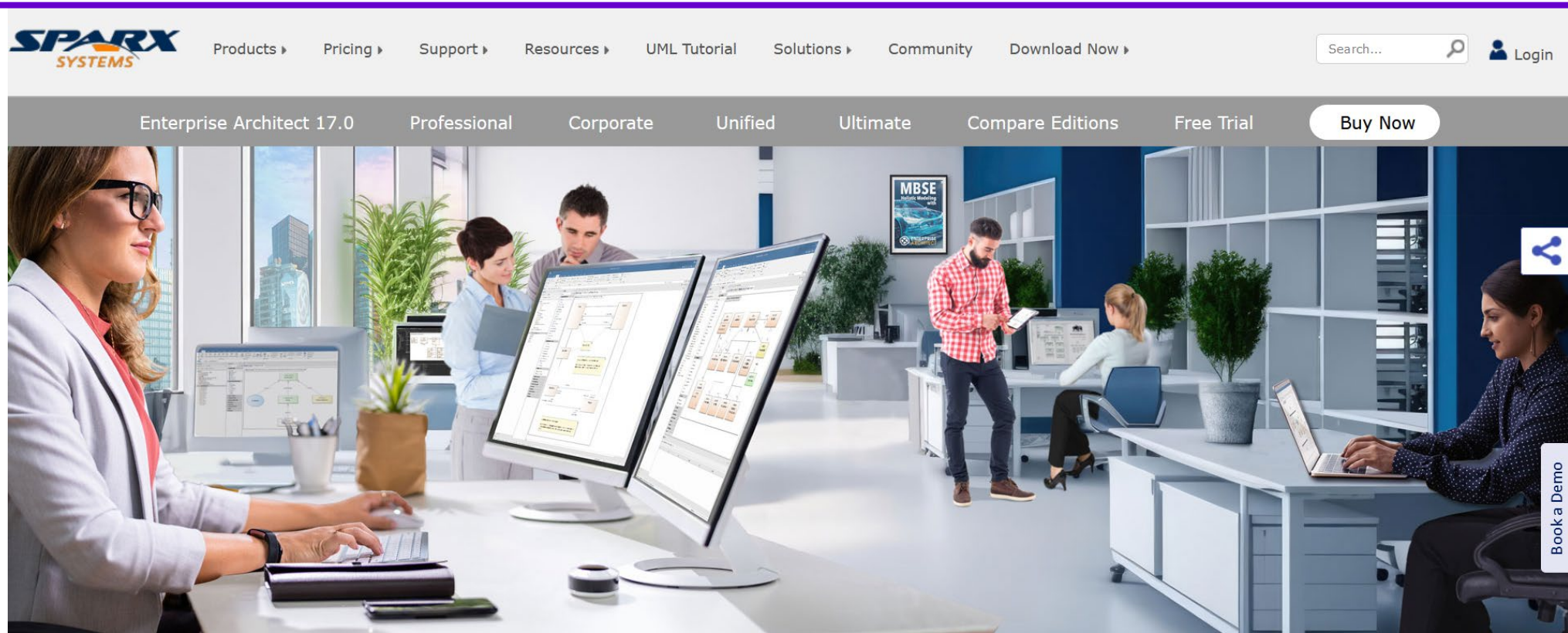
The #1 Development Tool Suite

that drives your project to success

A suite of design, analysis and management tools to drive your IT project development and digital transformation.



Specialized Design and Modeling Tools: SPARX Enterprise Architect



 ENTERPRISE ARCHITECT 17.0

Specialized Design and Modeling Tools: Diagram as Code Tools like Mermaid

```
from diagrams import Cluster, Diagram
from diagrams.aws.compute import ECS
from diagrams.aws.database import ElastiCache, RDS
from diagrams.aws.network import ELB
from diagrams.aws.network import Route53

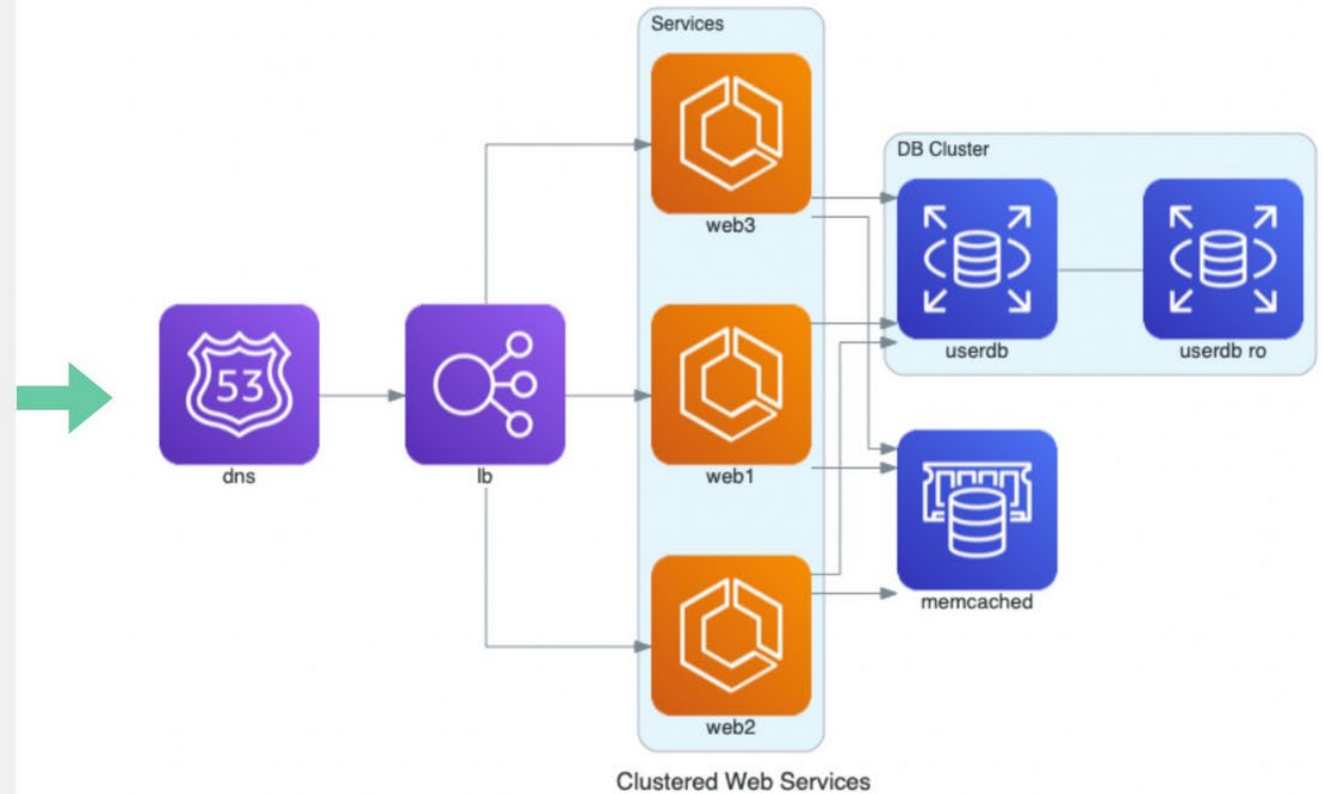
with Diagram("Clustered Web Services", show=False):
    dns = Route53("dns")
    lb = ELB("lb")

    with Cluster("Services"):
        svc_group = [ECS("web1"),
                     ECS("web2"),
                     ECS("web3")]

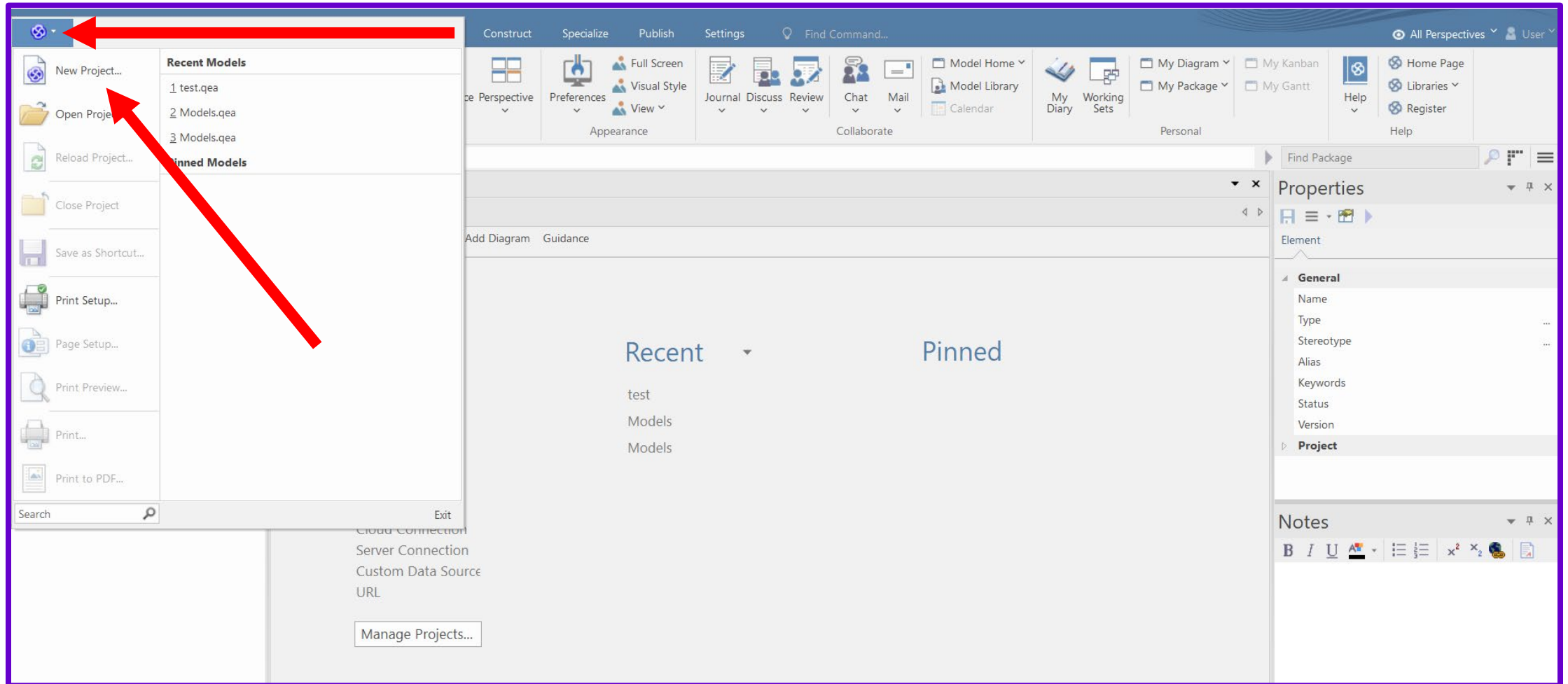
    with Cluster("DB Cluster"):
        db_primary = RDS("userdb")
        db_primary - [RDS("userdb ro")]

    memcached = ElastiCache("memcached")

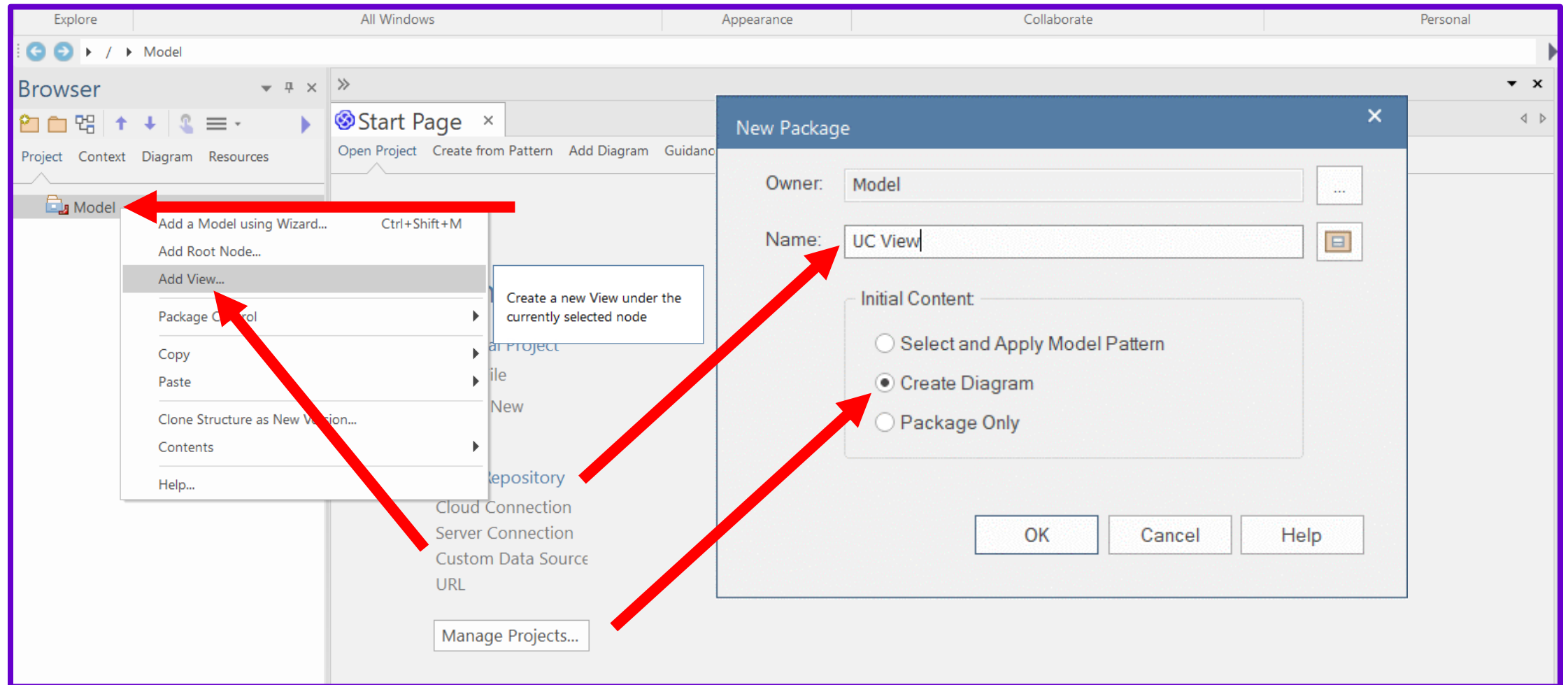
    dns >> lb >> svc_group
    svc_group >> db_primary
    svc_group >> memcached
```



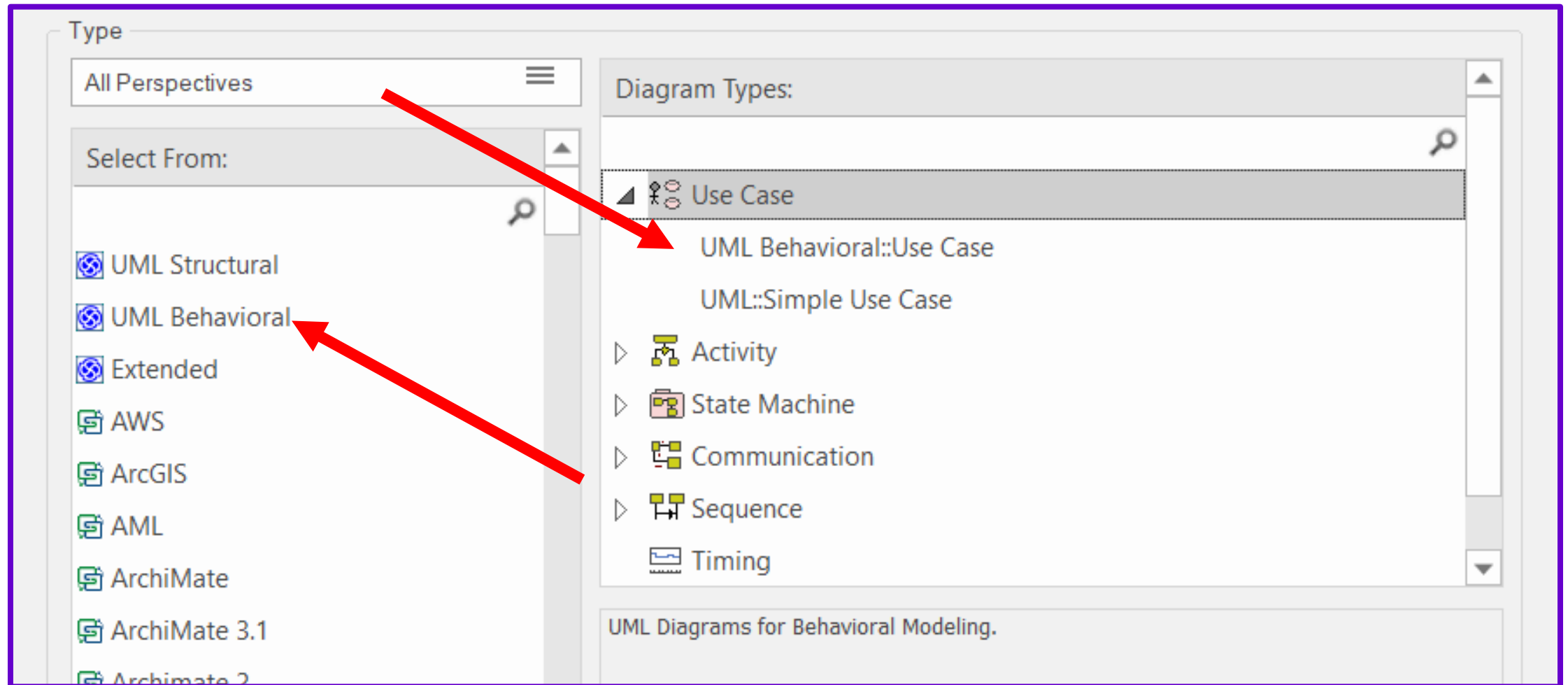
Step 1: Create new project



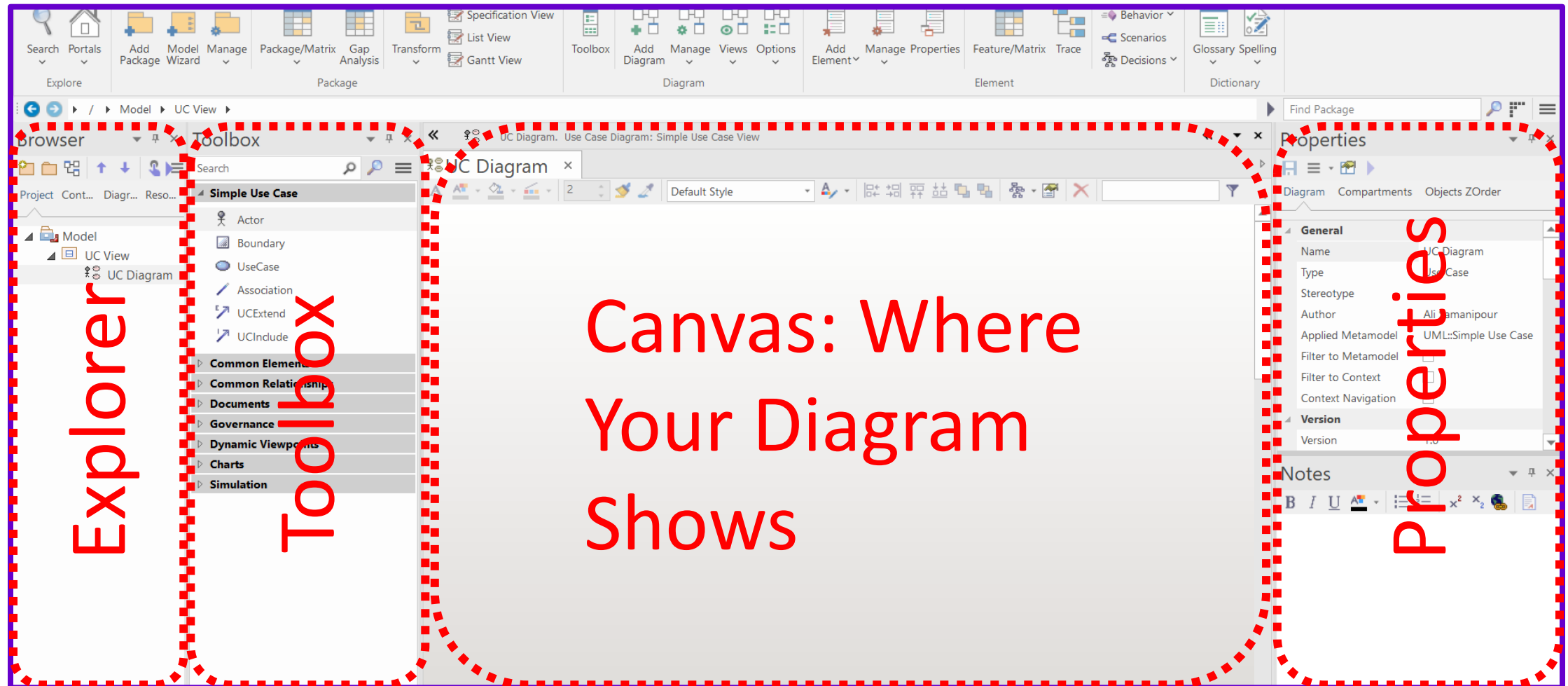
Step 2: Add View (Package) to Model



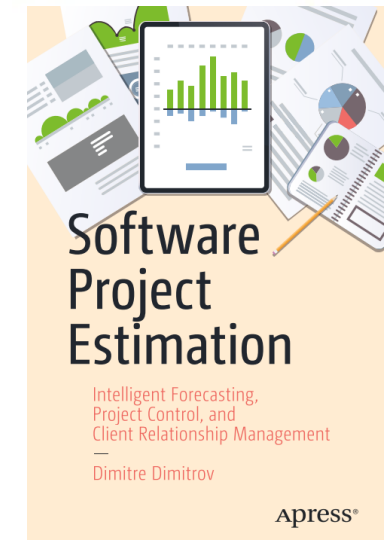
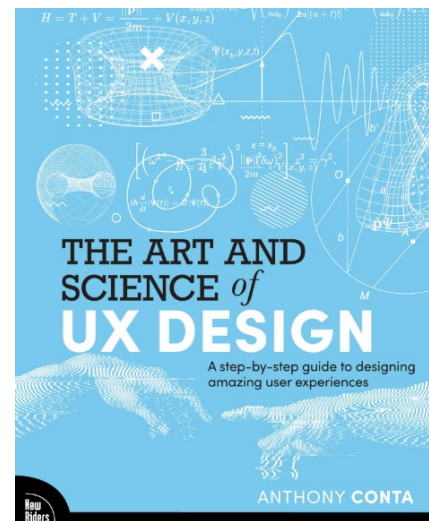
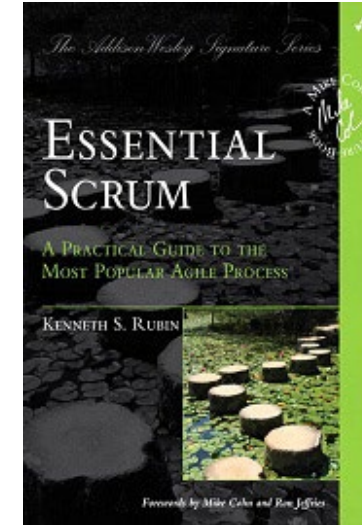
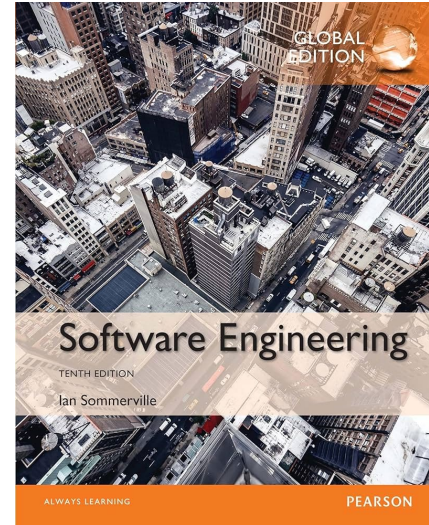
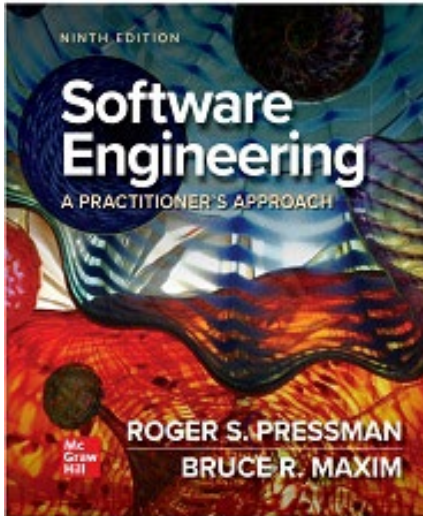
Step 3: Search & Select Diagram Type



Step 4: Start Modeling



Course References



Course References

- ***Designing User Interfaces***, Michal Malewicz & Diana Malewice, 2020
- ***UI Design Styles: Trends and Design Patterns***, Michal Malewicz & Diana Malewice, 2020
- ***What UX Is Really About :Introducing a Mindset for Great Experiences***, Celia Hodent, CRC Press, 2022
- ***Lean UX: Designing Great Products with Agile Teams 3rd Edition***, Jeff Gothelf & Josh Seiden, O'Reilly, 2021
- ***Laws of UX: Using Psychology to Design Better Products & Services***, Jon Yablonski, O'Reilly, 2020
- ***Designing and Prototyping Interfaces with Figma***, Fabio Staiano, Packet Publishing, 2022

Course References

- [1] Green, M. D. (2016). Scrum: Novice to Ninja: Methods for Agile, Powerful Development, SitePoint.
- [2] Ockerman, S. and S. Reindl (2019). Mastering professional scrum: A practitioner's guide to overcoming challenges and maximizing the benefits of agility, Addison-Wesley Professional.
- [3] Martin, R. C. (2019). Clean Agile, Pearson Education.
- [4] Hall, G. M. (2017). Adaptive Code: Agile coding with design patterns and SOLID principles, Microsoft
- [۵] سامانی پور، علی. (۲۰۱۸). آموزش اسکرام برای مدیریت چابک فرایند توسعه اپلیکیشن های وب و موبایل،
فرادرس